LeetCode Practice

25 Medium: Linked List

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

25
MEDIUM

Patterns (in order)

1. Linked List (25)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

25 questions | tap a title to jump to it

Medium questions (25)	Go >
Linked List (25)	**Go >**
M 1. Add Two Numbers	LC 2
M 2. Remove Nth Node From End of List	LC 19
M 3. Swap Nodes in Pairs	LC 24
M 4. Rotate List	LC 61
M 5. Remove Duplicates from Sorted List II	LC 82
M 6. Partition List	LC 86
M 7. Reverse Linked List II	LC 92
M 8. Convert Sorted List to Binary Search Tree	LC 109
M 9. Populating Next Right Pointers in Each Node	LC 116
M 10. Populating Next Right Pointers in Each Node II	LC 117
M 11. Copy List with Random Pointer	LC 138
M 12. LRU Cache	LC 146
M 13. Sort List	LC 148
M 14. Delete Node in a Linked List	LC 237
M 15. Odd Even Linked List	LC 328
M 16. Plus One Linked List	LC 369
M 17. Flatten a Multilevel Doubly Linked List	LC 430
M 18. Design Circular Queue	LC 622
M 19. Design Circular Deque	LC 641
M 20. Design Linked List	LC 707
M 21. Insert into a Sorted Circular Linked List	LC 708
M 22. Remove Zero Sum Consecutive Nodes from Linked List	LC 1171
M 23. Merge In Between Linked Lists	LC 1669
M 24. Delete the Middle Node of a Linked List	LC 2095
M 25. Insert Greatest Common Divisors in Linked List	LC 2807

Medium questions (25)

MEDIUM | TOPIC 1 OF 1

Linked List

25 Medium questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Add Two Numbers LC 2
2. Remove Nth Node From End of List LC 19
3. Swap Nodes in Pairs LC 24
4. Rotate List LC 61
5. Remove Duplicates from Sorted List II LC 82
6. Partition List LC 86
7. Reverse Linked List II LC 92
8. Convert Sorted List to Binary Search Tree LC 109
9. Populating Next Right Pointers in Each Node LC 116
10. Populating Next Right Pointers in Each Node II LC 117
11. Copy List with Random Pointer LC 138
12. LRU Cache LC 146
13. Sort List LC 148
14. Delete Node in a Linked List LC 237
15. Odd Even Linked List LC 328
16. Plus One Linked List LC 369
17. Flatten a Multilevel Doubly Linked List LC 430
18. Design Circular Queue LC 622
19. Design Circular Deque LC 641
20. Design Linked List LC 707
21. Insert into a Sorted Circular Linked List LC 708
22. Remove Zero Sum Consecutive Nodes from Linked List LC 1171
23. Merge In Between Linked Lists LC 1669
24. Delete the Middle Node of a Linked List LC 2095
25. Insert Greatest Common Divisors in Linked List LC 2807

MEDIUM | Linked List | LeetCode #2 | 1 of 25 | 1/25

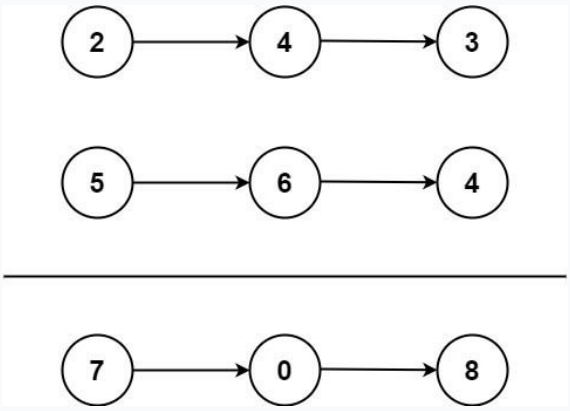
Add Two Numbers

QUESTION

You are given two non-empty linked lists representing two non-negative integers. The digits are stored in reverse order, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1	
Input	`l1 = [2,4,3], l2 = [5,6,4]`
Output	`[7,0,8]`
Explanation	*342 + 465 = 807.*



Example 2	
Input	`l1 = [0], l2 = [0]`
Output	`[0]`

Example 3	
Input	`l1 = [9,9,9,9,9,9,9], l2 = [9,9,9,9]`
Output	`[8,9,9,9,0,0,0,1]`

CONSTRAINTS

- The number of nodes in each linked list is in the range [1, 100].
- 0 <= Node.val <= 9
- It is guaranteed that the list represents a number that does not have leading zeros.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Two non-empty linked lists store digits of non-negative integers in REVERSE order.

Add the two numbers and return the sum as a reversed linked list. Right?"

PHASE 1 CLARIFY

"A few quick questions:

Can leading zeros appear in the input? Only for the number 0 itself.

Can the lists have different lengths? Yes — one might be much longer.

Should I handle carry past the longer list? Yes — e.g. 9+9 propagates."

"Let me walk: l1=[2,4,3] (342), l2=[5,6,4] (465). 342+465=807 -> [7,0,8] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
x = the integer formed by the first list's digits
y = the same for the second list
return the digits of x + y as a list
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Needs arbitrary-size integers (a 100-digit list overflows 64 bits) and converting long digit strings is quadratic. Adding digit by digit with a carry is O(max(n, m)) with no big numbers.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the big-int conversion — it breaks in fixed-width languages.

Simulate grade-school addition digit by digit with a carry variable.

Walk both lists simultaneously; at each step: total = d1 + d2 + carry.

New digit = total % 10, new carry = total // 10.

Continue until both lists are exhausted AND carry is 0.

Time: O(max(m,n)). Space: O(max(m,n)) for the result list.

Does that make sense before I code it?"

Time: O(n) — single traversal of the linked list. Optimal in-place.

Key insight: carry arithmetic simulates addition node-by-node without converting to integers — safe for any fixed-width language where big integers would overflow.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _2_AddTwoNumbers:
    def addTwoNumbers(self, l1, l2):
        dummy = cur = ListNode(0)
        carry = 0
```

```
        while l1 or l2 or carry:
            d1 = l1.val if l1 else 0
            d2 = l2.val if l2 else 0
            total = d1 + d2 + carry
            carry, digit = divmod(total, 10)
            cur.next = ListNode(digit)
            cur = cur.next
            if l1: l1 = l1.next
            if l2: l2 = l2.next
        return dummy.next
```

PHASE 5 TEST & DEBUG

"Let me trace addTwoNumbers([2,4,3],[5,6,4])."

Lists represent 342+465=807.

carry=0. Iterate both lists:

Node(2)+Node(5)+carry=0 -> sum=7. digit=7. New node 7. carry=0.

Node(4)+Node(6)+0 -> sum=10. digit=0. New node 0. carry=1.

Node(3)+Node(4)+1 -> sum=8. digit=8. New node 8. carry=0.

Both lists exhausted. carry=0 -> stop.

Result: 7->0->8 = 807 ok

[9,9,9,9,9,9,9]+[9,9,9,9]: carry=1 propagates through all digits.

After shorter list ends, continue with longer list + carry. Final carry=1 -> extra node.

WHY: pointer moves preserve the list's integrity; each rewiring step only changes local next-links while keeping the head/reference valid.

Build result: curr=dummy_head. curr.next=Node(carry+a+b). curr=curr.next. head->Node(7)->Node(0)->Node(8)->None. Return head.next ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(max(m,n)) process as many digits as the longer number plus at most one carry	**O(max(m,n))** result list

Follow-up: Add Two Numbers II (#445) — digits in forward order; use two stacks to reverse.

Follow-up: Multiply Strings (#43) — similar positional digit logic, O(n*m).

MEDIUM | Linked List | LeetCode #19 | 2 of 25 | 2/25

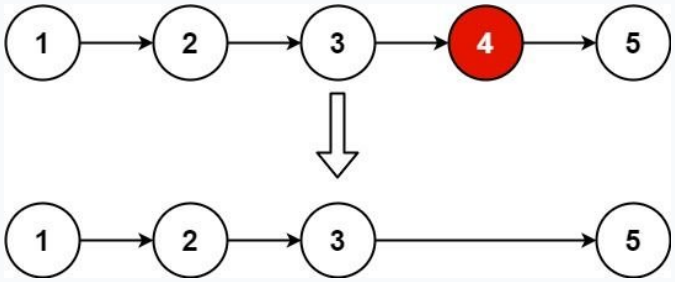
Remove Nth Node From End of List

QUESTION

Given the head of a linked list, remove the nth node from the end of the list and return its head.

Example 1

Input	`head = [1,2,3,4,5], n = 2`
Output	`[1,2,3,5]`



Example 2

Input	`head = [1], n = 1`
Output	`[]`

Example 3

Input	`head = [1,2], n = 1`
Output	`[1]`

CONSTRAINTS

- The number of nodes in the list is sz.
- 1 <= sz <= 30
- 0 <= Node.val <= 100
- 1 <= n <= sz
- Follow up: Could you do this in one pass?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Remove the nth node from the END of the list and return the modified head.

n is always valid. The follow-up asks for one pass. Right?"

"A few quick questions:

Can n equal the list length (removing the head)? Yes — handle with a dummy node.

Return the head, which may change if we remove it."

"Let me walk: [1,2,3,4,5], n=2 -> remove 4 -> [1,2,3,5] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
length = count the nodes
walk length - n - 1 steps from the head
unlink the next node
```

TIME	SPACE
O(L)	O(1)

NOTE

Not slow: two passes over L nodes. Moving two pointers with a gap of n finds the node in a single pass, which is the same O(L) with one traversal.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the two-pass approach — we can do it in one.

Advance a 'fast' pointer n+1 steps ahead of 'slow'.

Move both together until fast reaches None.

Then slow.next is the node to remove.

The dummy head ensures we never special-case removing the actual head.

Time: O(L). Space: O(1).

Does that make sense before I code it?

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _19_RemoveNthFromEnd:
    def removeNthFromEnd(self, head, n):
        dummy = ListNode(0, head)
        fast = slow = dummy
        for _ in range(n + 1):
            fast = fast.next
        while fast:
            fast = fast.next
            slow = slow.next
        slow.next = slow.next.next
        return dummy.next
```

PHASE 5 TEST & DEBUG

"Let me trace removeNthFromEnd([1,2,3,4,5], n=2)."

Use two pointers on list with dummy head. fast starts n+1 ahead.

dummy->1->2->3->4->5. fast=dummy. slow=dummy.

Move fast n+1=3 times: fast at node 3 (value 3).

Move both until fast.next=None: (3->4): fast at 4, slow at 2. (4->5): fast at 5, slow at 3. fast.next=None -> stop.

slow at node 3. slow.next=slow.next.next -> skip node 4.

List: dummy->1->2->3->5. Return dummy.next = 1->2->3->5 ok

PHASE 5 TEST & DEBUG

WHY fast leads by n+1: so slow lands at node BEFORE the one to delete.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(L)	**O(1)**
one pass	two pointer variables plus the dummy node

Dummy node is essential — without it, removing the head requires a special branch.

Follow-up: Middle of Linked List (#876) — same slow/fast pointer family.

Follow-up: if n could be invalid (> list length), add a length check before advancing.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Linked List | LeetCode #24 | 3 of 25 | 3/25

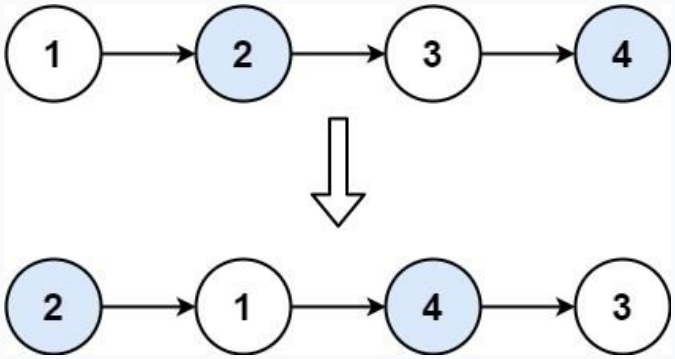
Swap Nodes in Pairs

QUESTION

Given a linked list, swap every two adjacent nodes and return its head. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed.)

Example 1

Input	`head = [1,2,3,4]`
Output	`[2,1,4,3]`



Example 2

Input	`head = []`
Output	`[]`

Example 3

Input	`head = [1]`
Output	`[1]`

Example 4

Input	`head = [1,2,3]`
Output	`[2,1,3]`

CONSTRAINTS

- The number of nodes in the list is in the range [0, 100].
- 0 <= Node.val <= 100

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Swap every two adjacent nodes and return the new head.

We swap the actual nodes — not just their values.

If there's an odd node at the end, leave it unchanged. Right?"

"A few quick questions:

Can the list be empty? Yes — return None.

O(1) space (no new nodes, just pointer rewiring)?"

PHASE 1 CLARIFY

"Let me walk: [1,2,3,4] -> [2,1,4,3] ok. [1,2,3] -> [2,1,3] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
put the nodes in an array
swap each neighbouring pair in the array
relink the nodes in array order
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear time but O(n) extra memory for the array. Rewiring next pointers pair by pair with a dummy head is O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) auxiliary list.

In-place pointer rewiring with a dummy predecessor:

For each pair (first, second): set prev.next=second, first.next=second.next, second.next=first.

Advance prev to first (now the second node in output) and repeat.

Time: O(n). Space: O(1).

Does that make sense before I code it?"

Key insight: the dummy head node eliminates the special-case handling for the first pair — the three-pointer rewiring pattern (prev->B->A->A.next) applies uniformly throughout.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _24_SwapNodesInPairs:
    def swapPairs(self, head):
        dummy = ListNode(0, head)
        prev = dummy
        while prev.next and prev.next.next:
            first = prev.next
            second = prev.next.next
            prev.next = second
            first.next = second.next
            second.next = first
            prev = first
        return dummy.next
```

PHASE 5 TEST & DEBUG

"Let me trace swapPairs(1->2->3->4)."

dummy->1->2->3->4. prev=dummy.

Iter 1: a=1,b=2. prev.next=2. a.next=b.next=3. b.next=a. -> dummy->2->1->3->4. prev=a=1. curr=3.

Iter 2: a=3,b=4. prev.next=4. a.next=4.next=None. b.next=a=3. -> 1.next=4->3->None. prev=3.

Result: 2->1->4->3 ok

WHY prev pointer: need to stitch previous pair's tail to current swapped head.

WHY 4 pointer variables: prev tracks tail of already-swapped section. Next pair becomes new a,b.

Edge: odd length 1->2->3: swap (1,2)=2->1. Node 3 is odd-one-out -> no swap. Result: 2->1->3 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
visit each node once	constant pointer variables

The dummy node eliminates special-casing the very first pair.

Follow-up: Reverse Nodes in k-Group (#25) — generalise to groups of k using the same logic.

Follow-up: recursive solution swaps the pair then recurses on head.next.next — clean but O(n) stack.

MEDIUM | Linked List | LeetCode #61 | 4 of 25 | 4/25

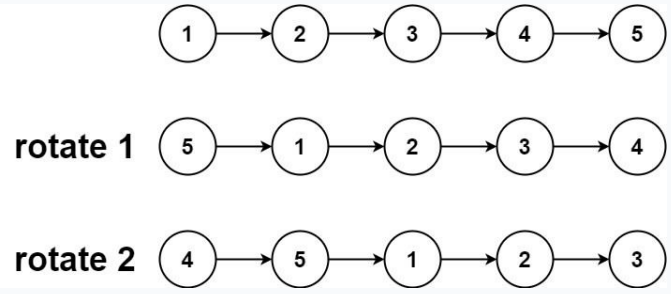
Rotate List

QUESTION

Given the head of a linked list, rotate the list to the right by k places.

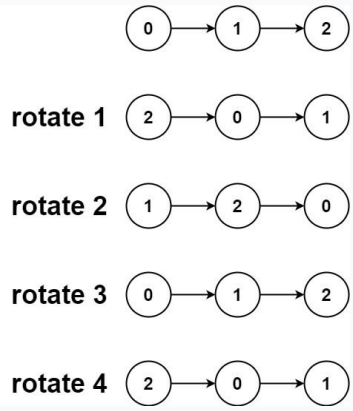
Example 1

Input	`head = [1,2,3,4,5], k = 2`
Output	`[4,5,1,2,3]`



Example 2

Input	`head = [0,1,2], k = 4`
Output	`[2,0,1]`



CONSTRAINTS

- The number of nodes in the list is in the range [0, 500].
- 100 <= Node.val <= 100
- 0 <= k <= $2 * 10^9$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Rotate the linked list to the right by k places — each rotation moves the tail to the front.

k can exceed the list length; rotations are modulo the length. Right?"

"A few quick questions:

Can k be 0? Yes — return unchanged. Can the list be empty? Yes — return None.

k up to 2*10^9 — so we definitely need the modulo trick."

PHASE 1 CLARIFY

"Let me walk: [1,2,3,4,5], k=2 -> [4,5,1,2,3] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for _ in range(k):
    move the last node to the front
```

TIME	SPACE
O(k * n)	O(1)

WHY IT IS SLOW

k can be huge, and each rotation walks the whole list. Rotating by k mod n and relinking once after finding the length and the new tail is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Approach: optimal approach

"The bottleneck is the O(n*k) brute — full cycles cancel so we only need k % length rotations.

Three steps in two passes:

1. Find length and tail by traversing once.
2. Effective k = k % length. If 0, return as-is.
3. New tail is at position (length - k - 1). New head is new_tail.next.

Connect old tail to old head and cut at new tail.

Time: O(n). Space: O(1).

Does that make sense before I code it?

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _61_RotateList:
    def rotateRight(self, head, k):
        if not head or not head.next or k == 0:
            return head
        tail, length = head, 1
        while tail.next:
            tail = tail.next
            length += 1
        k %= length
        if k == 0:
            return head
        new_tail = head
        for _ in range(length - k - 1):
            new_tail = new_tail.next
        new_head = new_tail.next
        new_tail.next = None
        tail.next = head
        return new_head
```

PHASE 5 TEST & DEBUG

"Let me trace rotateList(1->2->3->4->5, k=2)."

length=5. k=k%5=2. New tail at (5-2)=3rd node (index 2). New head at 4th node (index 3).

Find length, make circular: last.next=head.

Traverse to (5-2-1)=2nd node (index 2): node(3). new_head=node(3).next=node(4).

`node(3).next=None.`

Result: 4->5->1->2->3 ok

k=7 on [1..5]: k%5=2 -> same as above ok

WHY circular trick: rotate is just changing where the list starts/ends.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) one pass for length, one walk to new_tail	**O(1)**

The k%length shortcut is critical — without it k=10^9 on a 5-node list would loop forever.

Follow-up: Rotate Array (#189) — same modulo idea but uses three reversals on an array.

Follow-up: right-shift by 1 on a circular buffer is O(1) with a pointer — different trade-off.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Linked List | LeetCode #82 | 5 of 25 | 5/25

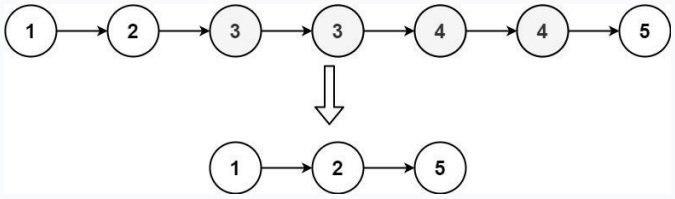
Remove Duplicates from Sorted List II

QUESTION

Given the head of a sorted linked list, delete all nodes that have duplicate numbers, leaving only distinct numbers from the original list. Return the linked list sorted as well.

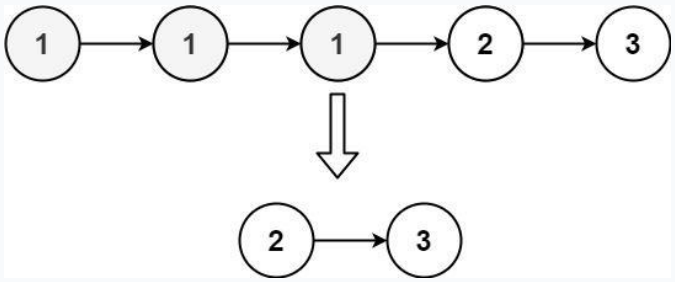
Example 1

Input	`head = [1,2,3,3,4,4,5]`
Output	`[1,2,5]`



Example 2

Input	`head = [1,1,1,2,3]`
Output	`[2,3]`



CONSTRAINTS

- The number of nodes in the list is in the range [0, 300].
- 100 <= Node.val <= 100
- The list is guaranteed to be sorted in ascending order.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Remove ALL nodes whose value appears more than once. Keep only nodes with distinct values. Right?"

"A few quick questions:

If a value appears 2+ times, ALL its nodes are deleted? Yes.

List is sorted — duplicates are always adjacent? Yes."

"Let me walk: [1,2,3,3,4,4,5] -> [1,2,5] ok. [1,1,1,2,3] -> [2,3] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
count = how many times each value occurs
build a new list from the values that occur once
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear but needs a map and a new list. The list is sorted, so one pass with a dummy head that skips whole runs of equal values is O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Two-pointer sentinel — O(n). Walk-through:"

`sentinel = ListNode(0); sentinel.next = head. prev = sentinel.`

While prev.next:

`curr = prev.next.`

If curr.next and curr.next.val == curr.val:

`val = curr.val.`

While prev.next and prev.next.val == val: prev.next = prev.next.next. <- skip all with this val

Else: prev = prev.next. <- no duplicate: advance prev

Return sentinel.next.

WHY sentinel: avoids special-casing removal of the head node — sentinel always points

to the true head, which may change if head itself is a duplicate.

WHY don't advance prev when skipping: we need to re-check prev.next after deletion

in case the next group is also a duplicate.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _82_RemoveDuplicatesFromSortedListII:
    def deleteDuplicates(self, head):
        dummy = ListNode(0, head)
        prev = dummy
        curr = head
        while curr:
            if curr.next and curr.val == curr.next.val:
                dup_val = curr.val
                while curr and curr.val == dup_val:
                    curr = curr.next
                prev.next = curr
            else:
                prev = curr
                curr = curr.next
        return dummy.next
```

PHASE 5 TEST & DEBUG

"Let me trace deleteDuplicatesII(1->2->3->3->4->4->5)."

dummy->1->2->3->3->4->4->5. prev=dummy. curr=1.

curr=1: curr.next=2!=1 -> not dup. prev=1. curr=2.

curr=2: curr.next=3!=2 -> prev=2. curr=3.

curr=3: curr.next=3==3 -> dup detected! Skip all with val=3: advance curr until curr.val!=3. curr becomes 4.

prev.next=4 (skip all 3s). curr=4.

curr=4: curr.next=4==4 -> dup! Skip all 4s. curr becomes 5. prev.next=5.

curr=5: curr.next=None -> prev=5.

Result: 1->2->5 ok

WHY: pointer moves preserve the list's integrity; each rewiring step only changes local next-links while keeping the head/reference valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
one pass	only pointer variables

Contrast with #83 (Remove Duplicates I) — #83 keeps one copy; #82 removes all.

Follow-up: unsorted list with duplicates to remove — need a set for O(n) time.

Follow-up: Remove Duplicates from Sorted Array II (#80) — array version, keep up to 2.

MEDIUM | Linked List | LeetCode #86 | 6 of 25 | 6/25

Partition List

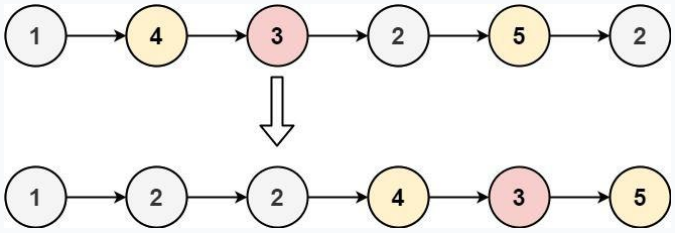
QUESTION

Given the head of a linked list and a value x, partition it such that all nodes less than x come before nodes greater than or equal to x.

You should preserve the original relative order of the nodes in each of the two partitions.

Example 1

Input	`head = [1,4,3,2,5,2], x = 3`
Output	`[1,2,2,4,3,5]`



Example 2

Input	`head = [2,1], x = 2`
Output	`[1,2]`

CONSTRAINTS

- The number of nodes in the list is in the range [0, 200].
- 100 <= Node.val <= 100
- 200 <= x <= 200

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Partition a linked list so nodes with val < x come before nodes with val >= x.

Preserve original relative order within each partition. Right?"

"A few quick questions:

Modify in-place? Yes. Stable partition (preserve order within each half)? Yes."

"Let me walk: head=[1,4,3,2,5,2], x=3 -> [1,2,2,4,3,5] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
less = values below x
rest = values >= x (both in original order)
rebuild the list from less then rest
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear but copies every value. Two dummy-headed lists linked in place (below x, and the rest), joined at the end, is O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Two dummy-headed lists, merge at end — O(n). Walk-through:"

before_dummy -> collects nodes with val < x.

after_dummy -> collects nodes with val >= x.

Scan original list:

If node.val < x: append to before list; before_tail = before_tail.next.

Else: append to after list; after_tail = after_tail.next.

After scan: before_tail.next = after_dummy.next (connect the two halves).

`after_tail.next = None (terminate the list).`

Return before_dummy.next.

WHY dummy nodes: avoids edge cases for empty before/after lists — always have a tail to append to.

Relative order preserved: nodes are appended in original order to each list.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _86_PartitionList:
    def partition(self, head, x):
        less_dummy = ListNode(0)
        greater_dummy = ListNode(0)
        less_tail = less_dummy
        greater_tail = greater_dummy
        curr = head
        while curr:
            if curr.val < x:
                less_tail.next = curr
                less_tail = curr
            else:
                greater_tail.next = curr
                greater_tail = curr
            curr = curr.next
        greater_tail.next = None
        less_tail.next = greater_dummy.next
        return less_dummy.next
```

PHASE 5 TEST & DEBUG

"Let me trace partition(1->4->3->2->5->2, x=3)."

Two dummy heads: less_head, greater_head.

less (< 3): 1->2->2. greater (>= 3): 4->3->5.

Append: less tail -> greater head. Sever greater tail.

Result: 1->2->2->4->3->5 ok

WHY two lists: maintain original relative order within each partition (stable partition).

Just relinking nodes — O(n) time, O(1) space.

Edge: all nodes less than x -> greater list empty. Sever greater_dummy.next=None. Append None correctly.

WHY two dummy heads: clean API — no special case for empty less or greater lists.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)** only two dummy nodes + tail pointers

Follow-up: partition by multiple pivots (like Dutch National Flag for linked lists).

Follow-up: Sort Colors (#75) — same partition idea for arrays.

MEDIUM | Linked List | LeetCode #92 | 7 of 25 | 7/25

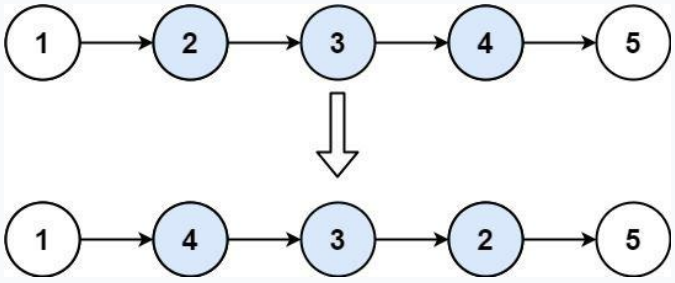
Reverse Linked List II

QUESTION

Given the head of a singly linked list and two integers left and right where left <= right, reverse the nodes of the list from position left to position right, and return the reversed list.

Example 1

Input	`head = [1,2,3,4,5], left = 2, right = 4`
Output	`[1,4,3,2,5]`



Example 2

Input	`head = [5], left = 1, right = 1`
Output	`[5]`

CONSTRAINTS

- The number of nodes in the list is n.
- 1 <= n <= 500
- 500 <= Node.val <= 500
- 1 <= left <= right <= n
- Follow up: Could you do it in one pass?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Reverse nodes from position left to right (1-indexed) in a linked list. Right?"

"A few quick questions:

1 <= left <= right <= n. Can left == right? Yes — no change."

"Let me walk: head=[1,2,3,4,5], left=2, right=4 -> [1,4,3,2,5] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
copy the values between positions left and right into
    an array
write them back reversed
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n)	O(right - left)

WHY IT IS SLOW

Linear, but needs extra memory for the segment. Reversing the sub-list in place by pointer flipping is O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Three-pointer reversal with four reconnects — O(n). Walk-through:"

Find the (left-1)-th node = prev. Let curr = prev.next (first node to reverse).

Reverse right-left+1 nodes:

For _ in right-left times:

`nxt = curr.next. curr.next = nxt.next. nxt.next = prev.next. prev.next = nxt.`

(Each iteration: detach nxt from after curr, re-insert it at the front of the reversed segment)

WHY this in-place reversal: instead of iterating backwards (which needs to know tail),

we repeatedly take the node after curr and insert it at the start of the reversed segment.

After k iterations: the segment is reversed, prev.next points to new head, curr points to new tail.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _92_ReverseLinkedListII:
    def reverseBetween(self, head, left, right):
        dummy = ListNode(0, head)
        pre = dummy
        for _ in range(left - 1):
            pre = pre.next
        cur = pre.next
        for _ in range(right - left):
            nxt = cur.next
            cur.next = nxt.next
            nxt.next = pre.next
            pre.next = nxt
        return dummy.next
```

PHASE 5 TEST & DEBUG

"Let me trace reverseBetween(1->2->3->4->5, left=2, right=4)."

Traverse to node before left: prev=1 (node at pos 1). curr=2 (pos 2).

Reverse (right-left)=2 times:

Step 1: nxt=3. curr(2).next=prev_rev=None. prev_rev=2. curr=3.

Step 2: nxt=4. curr(3).next=prev_rev=2. prev_rev=3. curr=4.

PHASE 5 TEST & DEBUG

Step 3 (right reached): nxt=5. curr(4).next=3. prev_rev=4. curr=5.

Stitch: node(1).next=4 (new sublist head). node(2).next=5 (curr, tail beyond right).

Result: 1->4->3->2->5 ok

WHY: pointer moves preserve the list's integrity; each rewiring step only changes local next-links while keeping the head/reference valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) walk to left + (right-left) swaps	**O(1)**

Follow-up: Reverse Nodes in k-Group (#25) — reverse every k nodes.

Follow-up: reverse even groups — apply this to every even-position group.

MEDIUM | Linked List | LeetCode #109 | 8 of 25 | 8/25

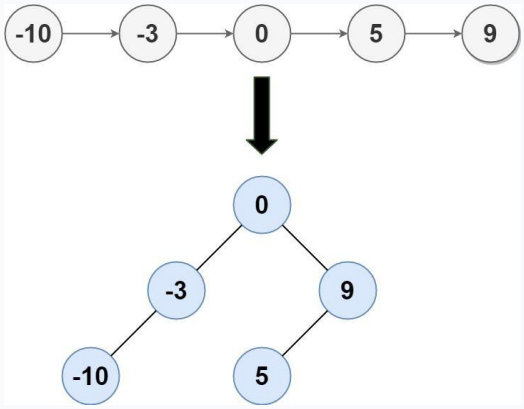
Convert Sorted List to Binary Search Tree

QUESTION

Given the head of a singly linked list where elements are sorted in ascending order, convert it to a height-balanced binary search tree.

Example 1

Input	`head = [-10,-3,0,5,9]`
Output	`[0,-3,9,-10,null,5]`
Explanation	*One possible answer is [0,-3,9,-10,null,5], which represents the shown height balanced BST.*



Example 2

Input	`head = []`
Output	`[]`

CONSTRAINTS

- The number of nodes in head is in the range [0, $2 * 10^4$].
- 10^5 <= Node.val <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Convert a sorted linked list to a height-balanced BST.

Height-balanced: every node's subtree heights differ by at most 1. Right?"

"A few quick questions:

Same as #108 but with a linked list instead of array? Yes — find the middle each time."

"Let me walk: head=[-10,-3,0,5,9] -> [0,-3,9,-10,null,5] or similar balanced BST ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
values = the list copied into an array
def build(lo, hi):
    pick the middle value as root, recurse on the
        two halves
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear, but copies the list into an array. Simulating an in-order traversal while building (so the list is consumed in order) uses only O(log n) extra space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Slow/fast pointer to find middle, then recursively build BST — O(n log n). Walk-through:"

`findMid(node): slow/fast pointers to find middle of linked list.`

Return slow (middle node), also disconnect: prev.next = None to split list.

`convert(head):`

If head is None: return None.

If head.next is None: return TreeNode(head.val).

`mid = findMid(head).`

`root = TreeNode(mid.val).`

`root.left = convert(head). <- left half of list (up to mid)`

`root.right = convert(mid.next). <- right half of list`

`return root.`

WHY middle as root: BST property + sorted list -> middle of current sublist = root gives balanced BST.

WHY disconnect list: separates the left and right halves for recursive calls.

Time: O(n log n). Space: O(log n) call stack.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _109_ConvertSortedListToBST:
    def sortedListToBST(self, head):
        if not head:
            return None
        if not head.next:
            return TreeNode(head.val)
        prev = None
        slow = fast = head
        while fast and fast.next:
            prev = slow; slow = slow.next; fast = fast.nex
t.next
```

```
        prev.next = None
        node = TreeNode(slow.val)
        node.left = self.sortedListToBST(head)
        node.right = self.sortedListToBST(slow.next)
        return node
```

PHASE 5 TEST & DEBUG

"Let me trace sortedListToBST on -10->-3->0->5->9."

`findMid(-10->...): slow ends at 0, disconnect: -10->-3 | 0->5->9. mid=0->root.`

Left subproblem: convert(-10->-3):

findMid: slow=-10, fast reaches end. mid=-3. disconnect: -10 | -3. mid=-3->root.

Left: convert(-10)->Node(-10). Right: convert(None)->None.

Right subproblem: convert(5->9): mid=9? No, findMid: slow=5,fast=9->next=None. mid=5->root. Right=convert(9)->Node(9).

Tree: 0(left=-3(left=-10), right=5(right=9)) ok

WHY slow/fast to find mid: splits list in half. Disconnecting isolates each half for recursion.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n)	**O(log n)**
O(log n) levels, each finding n midpoints total -> O(n log n)	recursion

Array approach: O(n) time, O(n) space — better time at cost of space.

Follow-up: Convert BST to Sorted Linked List (#426) — reverse operation.

MEDIUM | Linked List | LeetCode #116 | 9 of 25 | 9/25

Populating Next Right Pointers in Each Node

QUESTION

You are given a perfect binary tree where all leaves are on the same level, and every parent has two children. The binary tree has the following definition:

```
struct Node {
  int val;
  Node *left;
  Node *right;
  Node *next;
}
```

Populate each next pointer to point to its next right node. If there is no next right node, the next pointer should be set to NULL.

Initially, all next pointers are set to NULL.

Example 1

Input	`root = [1,2,3,4,5,6,7]`
Output	`[1,#,2,3,#,4,5,6,7,#]`
Explanation	*Given the above perfect binary tree (Figure A), your function should populate each next pointer to point to its next right node, just like in Figure B. The serialized output is in level order as connected by the next pointers, with '#' signifying the end of each level.*

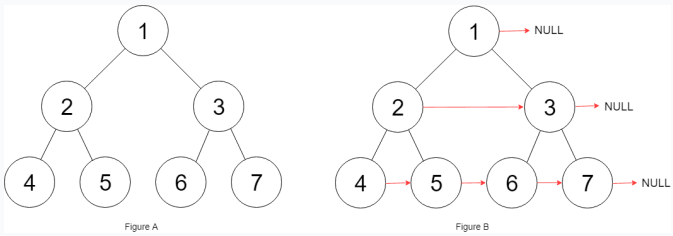


Figure A Figure B

Example 2

Input	`root = []`
Output	`[]`

CONSTRAINTS

- The number of nodes in the tree is in the range [0, 2^{12} - 1].
- 1000 <= Node.val <= 1000
- Follow-up:
- You may only use constant extra space.
- The recursive approach is fine. You may assume implicit stack space does not count as extra space for this problem.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Populate each node's next pointer to its right neighbour at the same level.

If no right neighbour, set next to NULL. Tree is a perfect binary tree. Right?"

"A few quick questions:

Perfect binary tree: all leaves at same level, all internal nodes have 2 children? Yes.

O(1) extra space? Yes per follow-up."

"Let me walk: root=[1,2,3,4,5,6,7] -> 4->5, 5->6, 6->7, 2->3, 1->NULL ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
queue = [root]
process level by level: link each node's next to the
    following node in the queue
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear, but the queue holds up to half the tree. In a perfect tree the already-built next pointers let you walk each level and link the next one with O(1) extra space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Level-order BFS, link adjacent nodes — O(n). Walk-through:"

Use the next pointers already set on the current level to traverse the next level:

`leftmost = root. While leftmost.left exists (not leaf level):`

`curr = leftmost.`

While curr:

`curr.left.next = curr.right. <- connect children of same parent`

If curr.next: curr.right.next = curr.next.left. <- connect across parents

`curr = curr.next.`

`leftmost = leftmost.left. <- go to next level`

WHY curr.next: traverses current level using next pointers (set in the previous iteration).

WHY perfect binary tree assumption (116): every node has two children -> simple pattern.

Time: O(n). Space: O(1) — no queue needed, uses existing next pointers.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _116_PopulatingNextRightPointers:
    def connect(self, root):
        if not root:
            return root
        leftmost = root
        while leftmost.left:
            head = leftmost
            while head:
                head.left.next = head.right
                if head.next:
                    head.right.next = head.next.left
                head = head.next
            leftmost = leftmost.left
        return root
```

PHASE 5 TEST & DEBUG

"Let me trace connect on perfect tree [1,2,3,4,5,6,7]."

leftmost=1. Level 1 (curr=1):

`curr.left.next = curr.right -> 2.next=3.`

curr.next=None -> no cross-parent link. curr=curr.next=None. leftmost=1.left=2.

Level 2 (curr=2):

2.left.next=2.right -> 4.next=5.

2.next=3 -> 2.right.next=2.next.left -> 5.next=3.left=6. curr=2.next=3.

3.left.next=3.right -> 6.next=7. 3.next=None. curr=None. leftmost=2.left=4.

Level 3: all leaves have next=None. Done. ok

Pointers: 2->3->null, 4->5->6->7->null ok. WHY use existing next pointers to traverse: O(1) space.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)** only pointers, no queue

Follow-up: Populating Next Right Pointers II (#117) — non-perfect tree; same idea but handle

variable number of children per node using a dummy head.

Follow-up: if we need to use the next pointers for level-order traversal later — they're set.

MEDIUM | Linked List | LeetCode #117 | 10 of 25 | 10/25

Populating Next Right Pointers in Each Node II

QUESTION

Given a binary tree

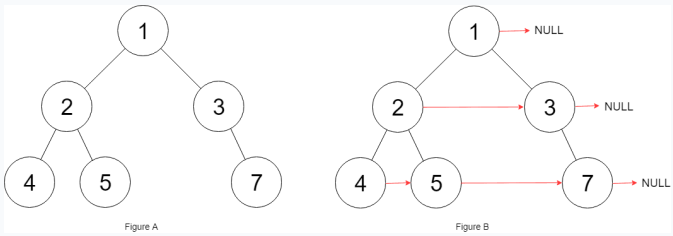
```
struct Node {
  int val;
  Node *left;
  Node *right;
  Node *next;
}
```

Populate each next pointer to point to its next right node. If there is no next right node, the next pointer should be set to NULL.

Initially, all next pointers are set to NULL.

Example 1

Input	`root = [1,2,3,4,5,null,7]`
Output	`[1,#,2,3,#,4,5,7,#]`
Explanation	*Given the above binary tree (Figure A), your function should populate each next pointer to point to its next right node, just like in Figure B. The serialized output is in level order as connected by the next pointers, with '#' signifying the end of each level.*



Example 2

Input	`root = []`
Output	`[]`

CONSTRAINTS

- The number of nodes in the tree is in the range [0, 6000].
- 100 <= Node.val <= 100
- Follow-up:
- You may only use constant extra space.
- The recursive approach is fine. You may assume implicit stack space does not count as extra space for this problem.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Same as #116 but for a general binary tree (not necessarily perfect).

Populate next pointers to the right neighbour at same level; NULL if rightmost. Right?"

PHASE 1 CLARIFY

"A few quick questions:

Must solve with O(1) extra space? Yes per follow-up.

Some nodes may have only one child? Yes."

"Let me walk: root=[1,2,3,4,5,null,7] -> 4->5->7, 2->3, 1->NULL ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
queue = [root]
process level by level: link each node's next to the
    following node in the queue
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear, but the queue can hold a whole level. Walking each level through its next pointers with a dummy head that threads the next level uses O(1) extra space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Level-order using next pointers (imperfect tree) — O(n). Walk-through:"

`dummy = ListNode(0). tail = dummy. curr = root.`

While curr: <- iterate through current level

If curr.left: tail.next = curr.left; tail = tail.next.

If curr.right: tail.next = curr.right; tail = tail.next.

`curr = curr.next. <- next node on current level`

If curr is None: <- reached end of current level

`curr = dummy.next. <- first node of next level`

`dummy.next = None; tail = dummy. <- reset for next level`

WHY dummy node: building next level's linked list without knowing the first node upfront.

WHY differs from Q116: tree may not be perfect -> must check if left/right exist.

Time: O(n). Space: O(1) — no extra queue.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _117_PopulatingNextRightPointersII:
    def connect(self, root):
        current = root
        while current:
            dummy = tail = type('Node', (), {'next': None}
)()
            node = current
```

```
            while node:
                if node.left:
                    tail.next = node.left
                    tail = tail.next
                if node.right:
                    tail.next = node.right
                    tail = tail.next
                node = node.next
            current = dummy.next
        return root
```

PHASE 5 TEST & DEBUG

"Let me trace connect on [1,2,3,4,5,null,7] (imperfect tree)."

dummy=ListNode(0), tail=dummy. curr=root=1.

Process level (curr=1): 1 has left=2->tail.next=2,tail=2. right=3->tail.next=3,tail=3.

curr=curr.next=None. curr=dummy.next=2. Reset: dummy.next=None, tail=dummy.

Process level (curr=2): 2.left=4->tail->4. 2.right=5->tail->5. curr=2.next=3.

3.left=None. 3.right=7->tail->7. curr=3.next=None. curr=dummy.next=4. Reset.

Process level (curr=4): 4,5,7 all leaves, no children. curr=None. Done.

Pointers: 2->3->null, 4->5->7->null ok. WHY dummy: unknown which is first child on next level.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)** only constant pointers

For perfect binary tree (#116), a simpler approach using left.next=right exists.

Follow-up: if levels are very wide, the O(1) space advantage matters significantly.

Follow-up: use this to implement level-order traversal without a queue.

MEDIUM | Linked List | LeetCode #138 | 11 of 25 | 11/25

Copy List with Random Pointer

QUESTION

A linked list of length n is given such that each node contains an additional random pointer, which could point to any node in the list, or null.

Construct a deep copy of the list. The deep copy should consist of exactly n brand new nodes, where each new node has its value set to the value of its corresponding original node. Both the next and random pointer of the new nodes should point to new nodes in the copied list such that the pointers in the original list and copied list represent the same list state. None of the pointers in the new list should point to nodes in the original list.

For example, if there are two nodes X and Y in the original list, where X.random --> Y, then for the corresponding two nodes x and y in the copied list, x.random --> y.

Return the head of the copied linked list.

The linked list is represented in the input/output as a list of n nodes. Each node is represented as a pair of [val, random_index] where:

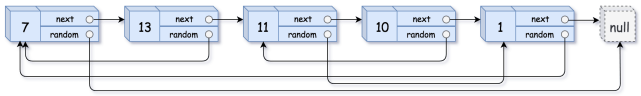
CONSTRAINTS

- val: an integer representing Node.val
- random_index: the index of the node (range from 0 to n-1) that the random pointer points to, or null if it does not point to any node.

Your code will only be given the head of the original linked list.

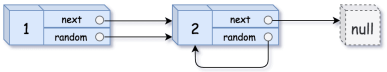
Example 1

Input	`head = [[7,null],[13,0],[11,4],[10,2],[1,0]]`
Output	`[[7,null],[13,0],[11,4],[10,2],[1,0]]`



Example 2

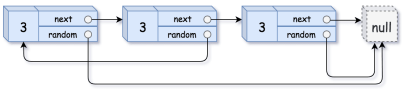
Input	`head = [[1,1],[2,1]]`
Output	`[[1,1],[2,1]]`



Example 3

Input	`head = [[3,null],[3,0],[3,null]]`
Output	`[[3,null],[3,0],[3,null]]`

QUESTION



- 0 <= n <= 1000
- 10^4 <= Node.val <= 10^4
- Node.random is null or is pointing to some node in the linked list.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Deep copy a linked list where each node has next AND a random pointer (can point anywhere or null).

Return the head of the new list, fully independent of the original. Right?"

"A few quick questions:

random can point to itself or be null? Yes — handle both."

"Let me walk: list with random pointers crossing -> produce an identical structure with new node objects ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
copies = one new node per original, kept in an array
for node in nodes:
    find the index of its random target by
        scanning the original list and link the
        copies
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Finding each random target costs O(n). A hash map from original to copy gives O(n) time and O(n) space; interleaving each copy after its original gives O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Two-pass to set random pointers — O(n). Walk-through:"

Pass 1: copy nodes without random/next. Use a map: original->copy.

Pass 2: for each original node: copy.next = map[original.next]; copy.random = map[original.random].

WHY two passes: in pass 1 we don't know if the random-pointed node's copy exists yet.

Pass 2 fills all pointers once all copies exist.

In-place O(1) space trick:

PHASE 3 OPTIMIZE

Interleave: A->A'->B->B'->... (insert copy after each original).

Set random: curr'.random = curr.random.next. <- curr.random is original; .next is copy

Separate: restore original list; extract copy list.

Time: O(n). Space: O(n) with map, O(1) with interleave.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _138_CopyListWithRandomPointer:
    def copyRandomList(self, head):
        if not head:
            return None
        old_to_new = {}
        current = head
        while current:
            old_to_new[current] = Node(current.val)
            current = current.next
        current = head
        while current:
            copy = old_to_new[current]
            copy.next = old_to_new.get(current.next)
            copy.random = old_to_new.get(current.random)
            current = current.next
        return old_to_new[head]
```

PHASE 5 TEST & DEBUG

"Let me trace copyRandomList([[7,null],[13,0],[11,4],[10,2],[1,0]])."

Two-pass with hash map (original->copy).

Pass 1: create all copy nodes. copies={orig_i: Node(val_i)}.

Pass 2: wire next and random pointers.

Node 0 (val=7, random=null): copies[0].next=copies[1], copies[0].random=None.

Node 1 (val=13, random=0): copies[1].next=copies[2], copies[1].random=copies[0].

... Continue for all nodes. Return copies[head] ok

O(1) space alt: interleave copies in original list, set randoms, then split.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** for the map (O(1) extra possible with list interleaving trick)

Follow-up: Clone Graph (#133) — same hashmap-of-clones pattern for arbitrary graphs.

Follow-up: implement the O(1)-space interleaving variant for an interview follow-up.

MEDIUM | Linked List | LeetCode #146 | 12 of 25 | 12/25

LRU Cache

QUESTION

Design a data structure that follows the constraints of a Least Recently Used (LRU) cache.

Implement the LRUCache class:

CONSTRAINTS

- LRUCache(int capacity) Initialize the LRU cache with positive size capacity.
- int get(int key) Return the value of the key if the key exists, otherwise return -1.
- void put(int key, int value) Update the value of the key if the key exists. Otherwise, add the key-value pair to the cache. If the number of keys exceeds the capacity from this operation, evict the least recently used key.

The functions get and put must each run in O(1) average time complexity.

- 1 <= capacity <= 3000
- 0 <= key <= 10^4
- 0 <= value <= 10^5
- At most 2 * 10^5 calls will be made to get and put.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Design an LRU cache with a fixed capacity: get(key)->value or -1, and put(key,value).

When capacity is exceeded, evict the least recently used item.

Both operations must be O(1) average time. Right?"

"A few quick questions:

Does get count as a use (making the item recently used)? Yes.

Does put update an existing key's value and recency? Yes."

"Let me walk capacity=2: put(1,1), put(2,2), get(1)->1 (1 is now MRU (Most Recently Used)),

`put(3,3) evicts 2, get(2)->-1 ok`

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
keep a list of (key, value) with the most recently
    used last
def get(k):
    search the list, if found move it to the end
        and return its value
def put(k, v):
    remove k if present, append it, and if over
        capacity drop the first item
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n) per operation	O(capacity)

WHY IT IS SLOW

Every operation searches or shifts a list. A hash map to doubly linked list nodes gives O(1) lookup, O(1) move-to-front and O(1) eviction.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) list shift on every access.

HashMap + Doubly Linked List gives O(1) for both:

HashMap: key -> DLL node (O(1) lookup).

DLL: maintains access order — head=LRU, tail=MRU.

get: move node to tail. put: add at tail; if over capacity, remove from head.

Use sentinel head/tail nodes to avoid edge-case pointer checks.

Time: O(1) all ops. Space: O(capacity).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _146_LRUCache:
    class _Node:
        def __init__(self, key=0, val=0):
            self.key = key; self.val = val; self.prev = se
lf.next = None

    def __init__(self, capacity):
        self.cap = capacity
        self.map = {}
        self.head = self._Node(); self.tail = self._Node()
        self.head.next = self.tail; self.tail.prev = self.
head

    def _remove(self, node):
        node.prev.next = node.next; node.next.prev = node.
prev

    def _insert_tail(self, node):
        node.prev = self.tail.prev; node.next = self.tail
        self.tail.prev.next = node; self.tail.prev = node

    def get(self, key):
        if key not in self.map: return -1
        node = self.map[key]
        self._remove(node); self._insert_tail(node)
        return node.val

    def put(self, key, value):
        if key in self.map: self._remove(self.map[key])
        node = self._Node(key, value)
        self.map[key] = node; self._insert_tail(node)
        if len(self.map) > self.cap:
            lru = self.head.next
            self._remove(lru); del self.map[lru.key]
```

PHASE 5 TEST & DEBUG

"Let me trace LRUCache(capacity=2): put(1,1),put(2,2),get(1),put(3,3),get(2)."

OrderedDict (or doubly linked list + hash map).

```
put(1,1): cache={1:1}. Order:[1].
put(2,2): cache={1:1,2:2}. Order:[1,2].
get(1): move 1 to end (most recent). Order:[2,1].
  Return 1 ok
put(3,3): capacity full. Evict LRU = front of order =
  2. cache={1:1,3:3}. Order:[1,3].
get(2): 2 not in cache -> return -1 ok
```

WHY LRU = front of OrderedDict: least recently used is at the "oldest" end.

```
move_to_end() on access keeps ordering correct in
  O(1).
```

Doubly linked list: head.next=LRU_node...MRU_node.next=tail. After get(1): move node(1) to MRU: node(1).prev.next=node(1).next; node(1).next=tail; tail.prev=node(1). head.next=node(2) now.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(1)	**O(capacity)**
all ops: HashMap O(1) lookup + DLL O(1) insert/remove	at most capacity nodes in the DLL and map

Python's OrderedDict implements this internally — valid in an interview to mention.

Follow-up: LFU Cache (#460) — evict least FREQUENTLY used; needs two hashmaps + DLL per frequency.

Follow-up: thread-safe LRU — wrap get/put with a threading.Lock.

MEDIUM | Linked List | LeetCode #148 | 13 of 25 | 13/25

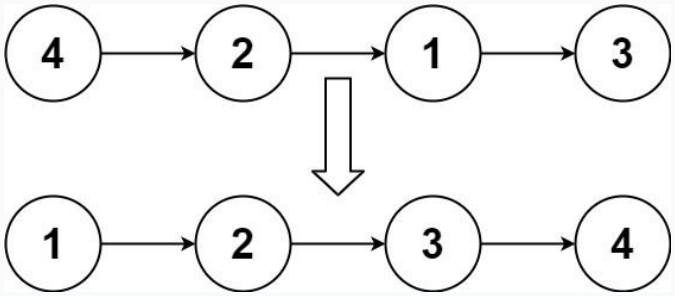
Sort List

QUESTION

Given the head of a linked list, return the list after sorting it in ascending order.

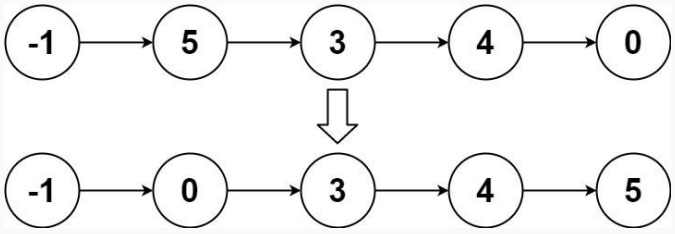
Example 1

Input	`head = [4,2,1,3]`
Output	`[1,2,3,4]`



Example 2

Input	`head = [-1,5,3,4,0]`
Output	`[-1,0,3,4,5]`



Example 3

Input	`head = []`
Output	`[]`

CONSTRAINTS

- The number of nodes in the list is in the range [0, 5 * 10^4].
- 10^5 <= Node.val <= 10^5
- Follow up: Can you sort the linked list in O(n logn) time and O(1) memory (i.e. constant space)?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Sort a linked list in ascending order.

The follow-up wants O(n log n) time and O(1) space. Right?"

"A few quick questions:

Values can be negative? Yes. Empty list is valid? Yes — return None."

"Let me walk: [4,2,1,3] -> [1,2,3,4] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
values = the list copied into an array
sort the array
write the values back into the nodes
```

TIME	SPACE
O(n log n)	O(n)

WHY IT IS SLOW

Right time, but O(n) extra memory. Merge sort directly on the linked list (find the middle, sort halves, merge by relinking) is O(n log n) with O(log n) stack, or O(1) space bottom-up.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) auxiliary list.

Bottom-up merge sort achieves O(n log n) time and O(1) space (no recursion stack):

Start by merging pairs of size 1, then pairs of size 2, then 4, and so on.

Each pass merges n/size pairs, and there are log n passes.

A dummy head simplifies relinking across rounds.

Time: O(n log n). Space: O(1).

Does that make sense before I code it?

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _148_SortList:
    def sortList(self, head):
        def merge(l1, l2):
            dummy = cur = ListNode(0)
            while l1 and l2:
                if l1.val <= l2.val: cur.next = l1; l1 = l
1.next
                else: cur.next = l2; l2 = l2.next
                cur = cur.next
            cur.next = l1 if l1 else l2
            return dummy.next

        def split(head, size):
            cur = head
            for _ in range(size - 1):
                if cur: cur = cur.next
            if not cur: return None, None
            rest = cur.next; cur.next = None
            return head, rest

        length, cur = 0, head
        while cur: length += 1; cur = cur.next

        size = 1
        while size < length:
            dummy = tail = ListNode(0)
```

```
            cur = head
            while cur:
                left, cur = split(cur, size)
                right, cur = split(cur, size)
                tail.next = merge(left, right)
                while tail.next: tail = tail.next
            head = dummy.next
            size *= 2
        return head
```

PHASE 5 TEST & DEBUG

"Let me trace sortList(4->2->1->3)."

Merge sort on linked list. Find mid (slow/fast): mid=2. Split: [4->2] and [1->3].

Sort [4->2]: mid=4. [4] and [2]. Merge: 2->4.

Sort [1->3]: mid=1. [1] and [3]. Merge: 1->3.

Merge [2->4] and [1->3]: 1<2->take 1. 2<3->take 2. 3<4->take 3. 4->take 4.

Result: 1->2->3->4 ok

WHY merge sort: O(n log n), O(log n) stack space (better than O(n) for array merge sort). Finding mid via slow/fast is O(n).

WHY bottom-up merge sort: O(1) extra space (no recursion stack). O(n log n) time guaranteed.

Step sizes: 1->2->4->8->... until step>=n. At each step, merge adjacent sublists of that size.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n)	**O(1)**
log n passes, each O(n)	no recursion stack

Top-down merge sort is simpler to write but uses O(log n) stack space.

Follow-up: Merge k Sorted Lists (#23) — use this merge step in a heap for O(n log k).

Follow-up: insertion sort for linked list (#147) — O(n²) but stable and in-place.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Linked List | LeetCode #237 | 14 of 25 | 14/25

Delete Node in a Linked List

QUESTION

There is a singly-linked list head and we want to delete a node node in it.

You are given the node to be deleted node. You will not be given access to the first node of head.

All the values of the linked list are unique, and it is guaranteed that the given node node is not the last node in the linked list.

Delete the given node. Note that by deleting the node, we do not mean removing it from memory. We mean:

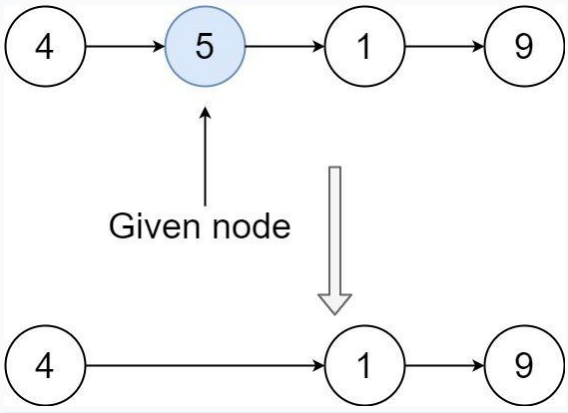
CONSTRAINTS

- The value of the given node should not exist in the linked list.
- The number of nodes in the linked list should decrease by one.
- All the values before node should be in the same order.
- All the values after node should be in the same order.

Custom testing:

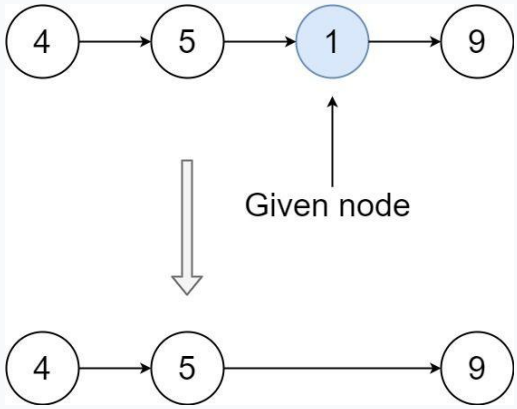
- For the input, you should provide the entire linked list head and the node to be given node. node should not be the last node of the list and should be an actual node in the list.
- We will build the linked list and pass the node to your function.
- The output will be the entire list after calling your function.

Example 1	
Input	`head = [4,5,1,9], node = 5`
Output	`[4,1,9]`
Explanation	*You are given the second node with value 5, the linked list should become 4 -> 1 -> 9 after calling your function.*



Example 2	
Input	`head = [4,5,1,9], node = 1`
Output	`[4,5,9]`
Explanation	*You are given the third node with value 1, the linked list should become 4 -> 5 -> 9 after calling your function.*

QUESTION



- The number of the nodes in the given list is in the range [2, 1000].
- 1000 <= Node.val <= 1000
- The value of each node in the list is unique.
- The node to be deleted is in the list and is not a tail node.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Delete a node given only that node (not the head). Copy the next node's value and skip next. Right?"

"A few quick questions:

The node to delete is guaranteed NOT to be the tail? Yes.

We don't have access to the previous node? Correct."

"Let me walk: list=[4,5,1,9], node=5 -> [4,1,9] ok

PHASE 2 BRUTE FORCE (also optimal)

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
to delete a given node with no access to the head:
    copy the next node's value into it and bypass the
    next node
```

TIME	SPACE
O(1)	O(1)

NOTE

Not slow: constant time. The only wrong approach is to look for the previous node, which needs the head (O(n)).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Copy next node's value, skip next — O(1). Walk-through:"

We can't access the previous node (singly linked list).

Instead: copy node.next.val into node.val, then skip node.next.

```
node.val = node.next.val <- this node 'becomes' the
  next node.
```

PHASE 3 OPTIMIZE

`node.next = node.next.next <- remove the (now duplicate) next node.`

Problem guarantees node is NOT the tail, so node.next always exists.

WHY this works: from the list's perspective, the node we 'deleted' is gone —

its value was overwritten and the pointer skips it.

Time: O(1). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _237_DeleteNodeInLinkedList:
    def deleteNode(self, node):
        node.val = node.next.val
        node.next = node.next.next
```

PHASE 5 TEST & DEBUG

"Let me trace the pointer rewiring steps and show how the list structure stays consistent."

list=[4,5,1,9], node=5: node.val=1 (copy next). node.next=9 (skip old 1 node).

list becomes [4,1,9] ok

list=[1,2], node=1: 1.val=2. 1.next=None. -> [2] ok

After the two operations, the node effectively takes on its successor's identity

WHY: pointer moves preserve the list's integrity; each rewiring step only changes local next-links while keeping the head/reference valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(1)	**O(1)** only two pointer assignments

This is a well-known trick — only works because node is guaranteed non-tail.

Follow-up: delete tail without head — impossible without a previous pointer.

Follow-up: Delete Middle Node (#2095) — given head, find and delete middle.

MEDIUM | Linked List | LeetCode #328 | 15 of 25 | 15/25

Odd Even Linked List

QUESTION

Given the head of a singly linked list, group all the nodes with odd indices together followed by the nodes with even indices, and return the reordered list.

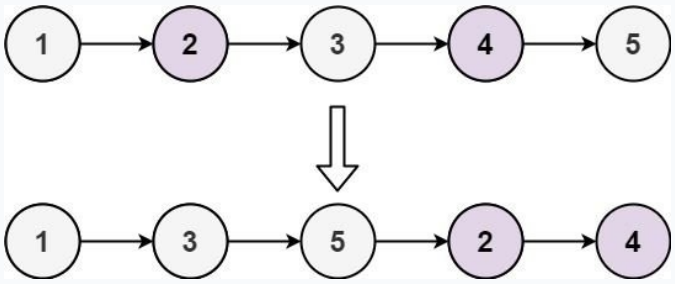
The first node is considered odd, and the second node is even, and so on.

Note that the relative order inside both the even and odd groups should remain as it was in the input.

You must solve the problem in O(1) extra space complexity and O(n) time complexity.

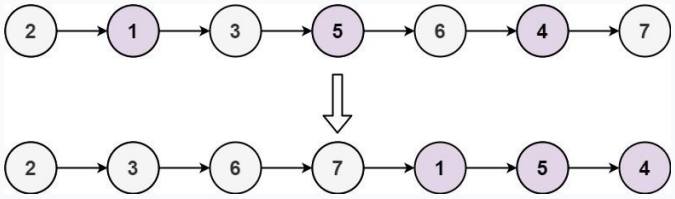
Example 1

Input	`head = [1,2,3,4,5]`
Output	`[1,3,5,2,4]`



Example 2

Input	`head = [2,1,3,5,6,4,7]`
Output	`[2,3,6,7,1,5,4]`



CONSTRAINTS

- The number of nodes in the linked list is in the range [0, 10^4].
- 10^6 <= Node.val <= 10^6

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Group all odd-position nodes first, then all even-position nodes.

1-indexed positions: node 1,3,5,... then 2,4,6,...

Preserve relative order within each group. O(1) space, O(n) time. Right?"

"A few quick questions:

Grouping is by POSITION not value? Yes — the problem is about indices.

Return the head of the modified list."

PHASE 1 CLARIFY

"Let me walk: [1,2,3,4,5] -> [1,3,5,2,4] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
odd = the values at odd positions in an array
even = the values at even positions
write odd then even back into the list
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear, but copies the values. Keep two chains (odd and even nodes), link them as you walk, and join the odd tail to the even head: O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) auxiliary lists.

In-place pointer manipulation with two sub-list heads:

`odd.next = odd.next.next (skip even nodes in the odd chain).`

`even.next = even.next.next (skip odd nodes in the even chain).`

At the end, link the tail of the odd chain to the head of the even chain.

Save even_head before the loop so we can link at the end.

Time: O(n). Space: O(1).

Does that make sense before I code it?

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _328_OddEvenLinkedList:
    def oddEvenList(self, head):
        if not head or not head.next:
            return head
        odd = head
        even = even_head = head.next
        while even and even.next:
            odd.next = even.next
            odd = odd.next
            even.next = odd.next
            even = even.next
        odd.next = even_head
        return head
```

PHASE 5 TEST & DEBUG

"Let me trace the pointer rewiring steps and show how the list structure stays consistent."

[1,2,3,4,5]: odd=1, even=2, even_head=2.

iter1: odd.next=3,odd=3. even.next=4,even=4.

iter2: odd.next=5,odd=5. even.next=None,even=None.

odd.next=even_head=2. -> 1->3->5->2->4->None ok

Even_head must be saved before the loop since we modify even's next

WHY: pointer moves preserve the list's integrity; each rewiring step only changes local next-links while keeping the head/reference valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
single pass	two pointer variables

Contrast with the brute that copies values — this reuses original nodes.

Follow-up: Partition List (#86) — partition by a value, not by position index.

Follow-up: Reorder List (#143) — interleave first and reversed second half.

MEDIUM | Linked List | LeetCode #369 | 16 of 25 | 16/25

Plus One Linked List

QUESTION

Given a non-negative integer represented as a linked list of digits, plus one to the integer.

The digits are stored such that the most significant digit is at the head of the list.

Example 1	
Input	`head = [1,2,3]`
Output	`[1,2,4]`

Example 2	
Input	`head = [0]`
Output	`[1]`

CONSTRAINTS

- The number of nodes in the linked list is in the range [1, 100].
- 0 <= Node.val <= 9
- The number represented by the linked list does not contain leading zeros except for the zero itself.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

A non-negative integer is stored as a linked list of digits, most significant first.

Add one to the number and return the updated list. Right?"

"A few quick questions:

Can all digits be 9 (e.g. 999+1=1000)? Yes — we'd need a new head node.

Can the number be 0? Yes — 0+1=1."

"Let me walk: [1,2,3] -> 123+1=124 -> [1,2,4] ok. [9,9,9] -> 999+1=1000 -> [1,0,0,0] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
turn the list into an integer
add 1
turn the result back into a list
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

The number can be arbitrarily long, so fixed-width integers overflow. Find the last digit that is not 9, add 1 there, and zero the 9s after it: O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the big-int round-trip that breaks in other languages.

Find the rightmost non-9 digit, increment it, and set all digits after it to 0.

If all digits are 9, prepend a new node with value 1 and zero out all existing nodes.

Use a dummy node with value 0 prepended to handle the all-9s case cleanly.

Time: O(n). Space: O(1) — in-place modification.

Does that make sense before I code it?"

Key insight: the rightmost non-9 digit is the only one that changes value — all trailing 9s become 0, and the all-9 case requires only a single new leading node.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _369_PlusOneLinkedList:
    def plusOne(self, head):
        dummy = ListNode(0)
        dummy.next = head
        not_nine = dummy
        cur = head
        while cur:
            if cur.val != 9: not_nine = cur
            cur = cur.next
        not_nine.val += 1
        cur = not_nine.next
        while cur:
            cur.val = 0; cur = cur.next
        return dummy.next if dummy.val == 0 else dummy
```

PHASE 5 TEST & DEBUG

"Let me trace the pointer rewiring steps and show how the list structure stays consistent."

[1,2,9]: not_nine ends at node(2). node(2).val=3. node(9).val=0. -> [1,3,0] ok

[9,9,9]: not_nine stays at dummy(0). dummy.val=0+1=1. 9->0,9->0,9->0.

dummy.val=1 -> return dummy -> [1,0,0,0] ok

Returning dummy when dummy.val!=0 handles the all-9s carry-out cleanly

PHASE 5 TEST & DEBUG

WHY: pointer moves preserve the list's integrity; each rewiring step only changes local next-links while keeping the head/reference valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) two passes (find not_nine, zero out tail)	**O(1)**

The dummy-node trick elegantly handles the all-9s case without a special branch.

Follow-up: Add Two Numbers (#2) — two lists, similar carry propagation.

Follow-up: Subtract 1 from a linked list — find rightmost non-0, decrement, set rest to 9.

MEDIUM | Linked List | LeetCode #430 | 17 of 25 | 17/25

Flatten a Multilevel Doubly Linked List

QUESTION

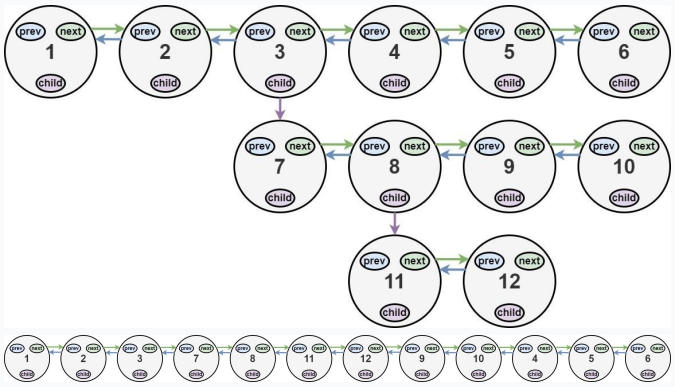
You are given a doubly linked list, which contains nodes that have a next pointer, a previous pointer, and an additional child pointer. This child pointer may or may not point to a separate doubly linked list, also containing these special nodes. These child lists may have one or more children of their own, and so on, to produce a multilevel data structure as shown in the example below.

Given the head of the first level of the list, flatten the list so that all the nodes appear in a single-level, doubly linked list. Let curr be a node with a child list. The nodes in the child list should appear after curr and before curr.next in the flattened list.

Return the head of the flattened list. The nodes in the list must have all of their child pointers set to null.

Example 1

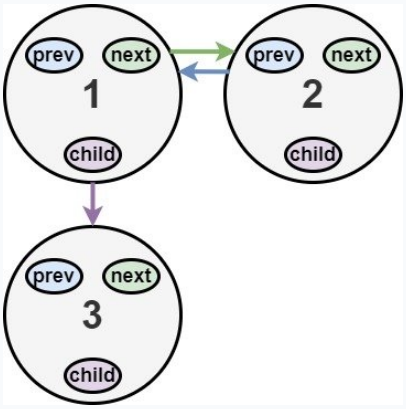
Input	`head = [1,2,3,4,5,6,null,null,null,7,8,9,10,null,null,11,12]`
Output	`[1,2,3,7,8,11,12,9,10,4,5,6]`
Explanation	*The multilevel linked list in the input is shown.* *After flattening the multilevel linked list it becomes:*



Example 2

Input	`head = [1,2,null,3]`
Output	`[1,3,2]`
Explanation	*The multilevel linked list in the input is shown.* *After flattening the multilevel linked list it becomes:*

QUESTION





Example 3

Input	`head = []`
Output	`[]`
Explanation	*There could be empty list in the input.*

CONSTRAINTS

- The number of Nodes will not exceed 1000.
- 1 <= Node.val <= 10^5
- How the multilevel linked list is represented in test cases:
- We use the multilevel linked list from Example 1 above:
- The serialization of each level is as follows:
- To serialize all levels together, we will add nulls in each level to signify no node connects to the upper node of the previous level. The serialization becomes:
- Merging the serialization of each level and removing trailing nulls we obtain:

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

A doubly linked list where nodes can have a child pointer to a sub-list.

Flatten it so all nodes appear in a single-level DLL in DFS pre-order

(node before its children, children before the rest of node's next). Right?"

"A few quick questions:

All child pointers must be set to null in the result? Yes.

1->2->3(child:7->8->9)->4: flattened -> 1->2->3->7->8->9->4 ok"

"Let me walk: 1-2-3-4-5-6, node 3 has child 7-8-9-10, node 8 has child 11-12.

PHASE 1 CLARIFY

Result: 1-2-3-7-8-11-12-9-10-4-5-6 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
nodes = the list of all nodes in the order of a
    flattening DFS
relink prev and next along the list and clear every
    child pointer
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear, but stores all nodes. Walking the list with a stack of pending 'next' nodes (splice the child list in when a child exists) is O(n) time and O(depth) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) auxiliary list.

Use a stack: when we encounter a node with a child, push its current next onto the stack,

insert the child inline, and continue.

When we reach a node with no next but the stack is non-empty, pop to resume the saved tail.

Time: O(n). Space: O(d) where d = max nesting depth.

Does that make sense before I code it?"

Key insight: the hash map provides O(1) lookup — a single pass over the data replaces the O(n) inner scan, reducing total complexity by an order of magnitude.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _430_FlattenMultilevel:
    def flatten(self, head):
        stack = []
        cur = head
        while cur:
            if cur.child:
                if cur.next: stack.append(cur.next)
                cur.next = cur.child
                cur.child.prev = cur
                cur.child = None
            elif not cur.next and stack:
                nxt = stack.pop()
                cur.next = nxt; nxt.prev = cur
            cur = cur.next
        return head
```

PHASE 5 TEST & DEBUG

"Let me trace the pointer rewiring steps and show how the list structure stays consistent."

1->2->3(child:7->8(child:11->12)->9->10)->4:

cur=3: has child. push next=4. 3.next=7, 7.prev=3. cur->7.

cur=8: has child. push next=9. 8.next=11, 11.prev=8. cur->11.

cur=12: no next, pop 9. 12.next=9, 9.prev=12. cur->9.

cur=10: no next, pop 4. 10.next=4, 4.prev=10. cur->4. Done.

-> 1-2-3-7-8-11-12-9-10-4-5-6 ok

Stack exactly mirrors the recursion call stack

WHY: pointer moves preserve the list's integrity; each rewiring step only changes local next-links while keeping the head/reference valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(d) stack depth = nesting depth

Iterative pre-order DFS — same pattern as Clone N-ary Tree but with prev-pointer maintenance.

Follow-up: unflatten the list — requires tracking where each child section started.

Follow-up: BFS order instead of DFS — use a queue instead of a stack.

MEDIUM | Linked List | LeetCode #622 | 18 of 25 | 18/25

Design Circular Queue

QUESTION

Design your implementation of the circular queue. The circular queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle, and the last position is connected back to the first position to make a circle. It is also called "Ring Buffer".

One of the benefits of the circular queue is that we can make use of the spaces in front of the queue. In a normal queue, once the queue becomes full, we cannot insert the next element even if there is a space in front of the queue. But using the circular queue, we can use the space to store new values.

Implement the MyCircularQueue class:

CONSTRAINTS

- MyCircularQueue(k) Initializes the object with the size of the queue to be k.
- int Front() Gets the front item from the queue. If the queue is empty, return -1.
- int Rear() Gets the last item from the queue. If the queue is empty, return -1.
- boolean enQueue(int value) Inserts an element into the circular queue. Return true if the operation is successful.
- boolean deQueue() Deletes an element from the circular queue. Return true if the operation is successful.
- boolean isEmpty() Checks whether the circular queue is empty or not.
- boolean isFull() Checks whether the circular queue is full or not.

You must solve the problem without using the built-in queue data structure in your programming language.

- 1 <= k <= 1000
- 0 <= value <= 1000
- At most 3000 calls will be made to enQueue, deQueue, Front, Rear, isEmpty, and isFull.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Design a circular queue (ring buffer) with a fixed capacity.

Support enQueue(val)->bool, deQueue()->bool, Front()->int, Rear()->int, isEmpty(), isFull().

Must not use Python's built-in queue. Right?"

"A few quick questions:

All enQueue/deQueue calls respect isEmpty/isFull — no need to guard against invalid calls? Yes."

"Let me walk: MyCircularQueue(3): enQueue(1,2,3)->T,T,T. enQueue(4)->F (full).

Rear()->3. isFull()->T. deQueue()->T. enQueue(4)->T. Rear()->4 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
store the items in an array with the front at index 0
deQueue:
    remove index 0 and shift all other items left
```

TIME	SPACE
O(n) per dequeue	O(k)

WHY IT IS SLOW

Shifting costs O(n) for every dequeue. A circular buffer with head and size indices (wrapping with modulo) makes enQueue and deQueue O(1).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) pop(0) in deQueue.

Use a fixed-size array with head/tail pointers and modular arithmetic.

enQueue: data[tail] = val; tail = (tail+1) % k; size++.

deQueue: head = (head+1) % k; size--.

Front: data[head]. Rear: data[(tail-1) % k].

All ops O(1). Space O(k).

Does that make sense before I code it?"

Key insight: a fixed-size array with head/tail indices and modular arithmetic (tail = (tail+1) % k) wraps the queue in O(1) without shifting — no dynamic allocation needed.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _622_DesignCircularQueue:
    def __init__(self, k):
        self._data = [0] * k
        self._head = self._size = 0
        self._cap = k

    def enQueue(self, val):
        if self.isFull(): return False
        self._data[(self._head + self._size) % self._cap]
= val
        self._size += 1
        return True

    def deQueue(self):
        if self.isEmpty(): return False
        self._head = (self._head + 1) % self._cap
        self._size -= 1
        return True

    def Front(self): return -1 if self.isEmpty() else self
._data[self._head]
    def Rear(self): return -1 if self.isEmpty() else self.
```

```
_data[(self._head + self._size - 1) % self._cap]
    def isEmpty(self): return self._size == 0
    def isFull(self): return self._size == self._cap
```

PHASE 5 TEST & DEBUG

"Let me trace MyCircularQueue(k=3): enQueue(1),enQueue(2), enQueue(3),enQueue(4),Rear(),isFull(),deQueue(),enQueue(4), Rear()."

Array of size 3. head=0, tail=0, size=0.

enQueue(1): arr[0]=1,tail=1,size=1. enQueue(2): arr[1]=2,tail=2,size=2.

enQueue(3): arr[2]=3,tail=0,size=3. (wraps around)

enQueue(4): size==k=3 -> False ok

Rear(): arr[(tail-1+k)%k]=arr[2]=3 ok

isFull(): size==k=3 -> True ok

deQueue(): head=(0+1)%3=1, size=2.

enQueue(4): arr[tail=0]=4, tail=1, size=3.

Rear(): arr[(1-1+3)%3]=arr[0]=4 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

All ops O(1). Space O(k).

The modulo wraps head/tail around the fixed array — that is the 'circular' part.

Follow-up: Design Circular Deque (#641) — add enqueueFront/dequeueRear using same technique.

Follow-up: thread-safe circular buffer — add a threading.Lock around enQueue/deQueue.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Linked List | LeetCode #641 | 19 of 25 | 19/25

Design Circular Deque

QUESTION

Design your implementation of the circular double-ended queue (deque).

Implement the MyCircularDeque class:

CONSTRAINTS

- MyCircularDeque(int k) Initializes the deque with a maximum size of k.
- boolean insertFront() Adds an item at the front of Deque. Returns true if the operation is successful, or false otherwise.
- boolean insertLast() Adds an item at the rear of Deque. Returns true if the operation is successful, or false otherwise.
- boolean deleteFront() Deletes an item from the front of Deque. Returns true if the operation is successful, or false otherwise.
- boolean deleteLast() Deletes an item from the rear of Deque. Returns true if the operation is successful, or false otherwise.
- int getFront() Returns the front item from the Deque. Returns -1 if the deque is empty.
- int getRear() Returns the last item from Deque. Returns -1 if the deque is empty.
- boolean isEmpty() Returns true if the deque is empty, or false otherwise.
- boolean isFull() Returns true if the deque is full, or false otherwise.
- 1 <= k <= 1000
- 0 <= value <= 1000
- At most 2000 calls will be made to insertFront, insertLast, deleteFront, deleteLast, getFront, getRear, isEmpty, isFull.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Design a circular deque supporting insertFront, insertLast, deleteFront, deleteLast, getFront, getRear. Right?"

"A few quick questions:

All within O(1) time? Yes. Return false if deque is full/empty? Yes."

"Let me walk: MyCircularDeque(3): insertLast(1)->T, insertLast(2)->T, insertFront(3)->T, insertFront(4)->F (full) ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
keep the items in an array
insertFront / deleteFront: shift every other item by
    one place
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n) per front operation	O(k)

WHY IT IS SLOW

Operations at the front shift the array. A circular buffer with front and rear indices (wrapping with modulo) makes all four end operations O(1).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Doubly linked list with head/tail sentinels — O(1) per operation. Walk-through:"

Maintain a doubly linked list. head sentinel nodes sentinel tail.

insertFront: create node, link between head sentinel and first real node.

insertLast: create node, link between last real node and tail sentinel.

deleteFront: unlink first real node (head.next). Return its val.

deleteLast: unlink last real node (tail.prev). Return its val.

getFront, getRear: peek at head.next and tail.prev.

WHY sentinel nodes: eliminate edge cases for empty list — head.next and tail.prev

always exist (pointing to each other when empty -> isEmpty = head.next == tail).

Time: O(1) all operations. Space: O(capacity).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _641_DesignCircularDeque:
    def __init__(self, k):
        self._data = [0] * k
        self._front = 0
        self._size = 0
        self._cap = k

    def insertFront(self, value):
        if self.isFull(): return False
        self._front = (self._front - 1) % self._cap
        self._data[self._front] = value
        self._size += 1
        return True

    def insertLast(self, value):
        if self.isFull(): return False
        self._data[(self._front + self._size) % self._cap]
 = value
        self._size += 1
        return True

    def deleteFront(self):
        if self.isEmpty(): return False
        self._front = (self._front + 1) % self._cap
        self._size -= 1
        return True
```

```
    def deleteLast(self):
        if self.isEmpty(): return False
        self._size -= 1
        return True

    def getFront(self): return -1 if self.isEmpty() else s
elf._data[self._front]
    def getRear(self): return -1 if self.isEmpty() else se
lf._data[(self._front + self._size - 1) % self._cap]
    def isEmpty(self): return self._size == 0
    def isFull(self): return self._size == self._cap
```

PHASE 5 TEST & DEBUG

"Let me trace MyCircularDeque(k=3): insertLast(1),insertLast(2),insertFront(3),insertFront(4),getRear()."

Doubly linked list or circular array. Use array: arr=[_,_,_], front=1, rear=0, size=0.

insertLast(1): rear=(0+1)%3=1, arr[1]=1, size=1.

insertLast(2): rear=2, arr[2]=2, size=2.

insertFront(3): front=(1-1+3)%3=0, arr[0]=3, size=3.

insertFront(4): size==k=3 -> False ok (full).

getRear(): arr[rear=2]=2 ok

getBack: arr[(rear-size? No): arr[rear]=2 where rear points to last inserted rear element ok

Circular deque=[_,1,_,3]: front=1, rear=3. insertFront(5): deque=[5,1,_,3]. deleteLast(): deque=[5,1,_,_]. deque=[(front->)5,1,...] implemented as fixed array with head/tail pointers ok

PHASE 6 COMPLEXITY & FOLLOW-UP

All ops O(1). Space O(k).

Follow-up: Design Circular Queue (#622) — single-ended version of this.

Follow-up: monotonic deque uses this structure for sliding window problems.

MEDIUM | Linked List | LeetCode #707 | 20 of 25 | 20/25

Design Linked List

QUESTION

Design your implementation of the linked list. You can choose to use a singly or doubly linked list. A node in a singly linked list should have two attributes: val and next. val is the value of the current node, and next is a pointer/reference to the next node. If you want to use the doubly linked list, you will need one more attribute prev to indicate the previous node in the linked list. Assume all nodes in the linked list are 0-indexed.

Implement the MyLinkedList class:

CONSTRAINTS

- MyLinkedList() Initializes the MyLinkedList object.
- int get(int index) Get the value of the indexth node in the linked list. If the index is invalid, return -1.
- void addAtHead(int val) Add a node of value val before the first element of the linked list. After the insertion, the new node will be the first node of the linked list.
- void addAtTail(int val) Append a node of value val as the last element of the linked list.
- void addAtIndex(int index, int val) Add a node of value val before the indexth node in the linked list. If index equals the length of the linked list, the node will be appended to the end of the linked list. If index is greater than the length, the node will not be inserted.
- void deleteAtIndex(int index) Delete the indexth node in the linked list, if the index is valid.
- 0 <= index, val <= 1000
- Please do not use the built-in LinkedList library.
- At most 2000 calls will be made to get, addAtHead, addAtTail, addAtIndex and deleteAtIndex.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Design a singly linked list supporting: get(index), addAtHead(val), `addAtTail(val), addAtIndex(index, val), deleteAtIndex(index).`

Indices are 0-based. addAtIndex at exactly size appends; beyond size is ignored. Right?"

"A few quick questions:

get/deleteAtIndex with invalid index return -1 / do nothing? Yes."

"Let me walk: addAtHead(1)->[1]. addAtTail(3)->[1,3]. addAtIndex(1,2)->[1,2,3].

`get(1)->2. deleteAtIndex(1)->[1,3]. get(1)->3 ok`

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
store the values in an array
def get(i):
    return the item
    addAtIndex / deleteAtIndex: shift the items
        after the index
```

TIME	SPACE
O(n) for add and delete	O(n)

WHY IT IS SLOW

Array shifts make inserts and deletes O(n). A linked list with a dummy head (and a tail pointer) makes the ends O(1), though reaching an index is still O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) list shift on head insertions/deletions.

A true singly linked list with a sentinel dummy head and size counter:

all operations walk to the (index-1)th node and rewire from there.

No shifting needed — pointer operations only.

Time: O(index) per op — O(n) worst case, O(1) at head.

Space: O(n) total nodes."

Key insight: the hash map provides O(1) lookup — a single pass over the data replaces the O(n) inner scan, reducing total complexity by an order of magnitude.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _707_MyLinkedList:
    class _Node:
        def __init__(self, val=0): self.val = val; self.ne
xt = None

    def __init__(self):
        self._dummy = self._Node()
        self._size = 0

    def _get_node(self, index):
        cur = self._dummy
        for _ in range(index + 1): cur = cur.next
        return cur

    def get(self, index):
        if index < 0 or index >= self._size: return -1
        return self._get_node(index).val

    def addAtHead(self, val): self.addAtIndex(0, val)
    def addAtTail(self, val): self.addAtIndex(self._size,
val)
```

```python
    def addAtIndex(self, index, val):
        if index > self._size: return
        prev = self._dummy
        for _ in range(index): prev = prev.next
        node = self._Node(val)
        node.next = prev.next; prev.next = node
        self._size += 1

    def deleteAtIndex(self, index):
        if index < 0 or index >= self._size: return
        prev = self._dummy
        for _ in range(index): prev = prev.next
        prev.next = prev.next.next
        self._size -= 1
```

PHASE 5 TEST & DEBUG

"Let me trace MyLinkedList: addAtHead(1), addAtTail(3), addAtIndex(1,2), get(1)."

Use dummy head sentinel for easier insertion

addAtHead(1): dummy->1. Size=1

addAtTail(3): dummy->1->3. Size=2

addAtIndex(1,2): traverse 1 step from dummy -> at node(1). Insert after: dummy->1->2->3. Size=3

get(1): traverse 1+1=2 steps from dummy -> node(2) -> return 2 ok

WHY dummy head: no special case for head insertions/deletions; uniform traversal logic.

Edge: addAtIndex(size,val) = addAtTail; addAtIndex(>size,val) = do nothing.

PHASE 6 COMPLEXITY & FOLLOW-UP

get/add/delete: O(index) — O(n) worst case. addAtHead: O(1).

Space O(n) for n stored nodes.

Follow-up: doubly linked list — add a prev pointer for O(1) head and tail ops.

Follow-up: skip list gives O(log n) average for all ops — classic interview extension.

MEDIUM | Linked List | LeetCode #708 | 21 of 25 | 21/25

Insert into a Sorted Circular Linked List

QUESTION

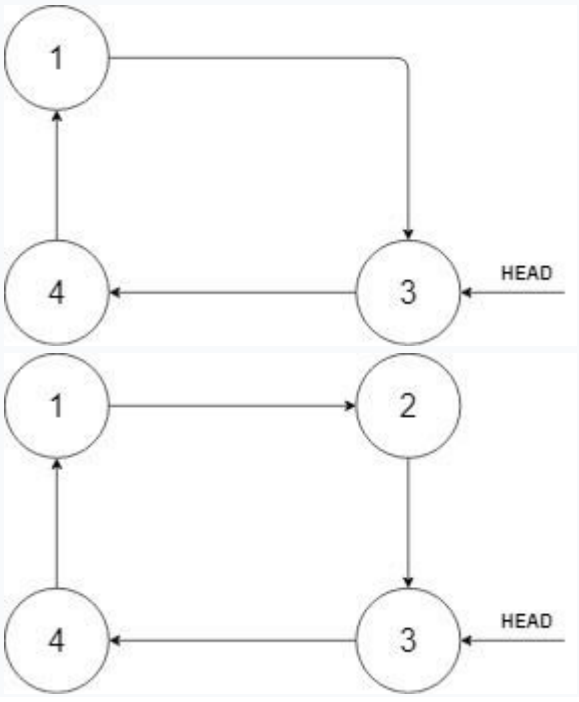
Given a Circular Linked List node, which is sorted in non-descending order, write a function to insert a value insertVal into the list such that it remains a sorted circular list. The given node can be a reference to any single node in the list and may not necessarily be the smallest value in the circular list.

If there are multiple suitable places for insertion, you may choose any place to insert the new value. After the insertion, the circular list should remain sorted.

If the list is empty (i.e., the given node is null), you should create a new single circular list and return the reference to that single node. Otherwise, you should return the originally given node.

Example 1

Input	`head = [3,4,1], insertVal = 2`
Output	`[3,4,1,2]`
Explanation	*In the figure above, there is a sorted circular list of three elements. You are given a reference to the node with value 3, and we need to insert 2 into the list. The new node should be inserted between node 1 and node 3. After the insertion, the list should look like this, and we should still return node 3.*



QUESTION

Example 2

Input	`head = [], insertVal = 1`
Output	`[1]`
Explanation	*The list is empty (given head is null). We create a new single circular list and return the reference to that single node.*

Example 3

Input	`head = [1], insertVal = 0`
Output	`[1,0]`

CONSTRAINTS

- The number of nodes in the list is in the range $[0, 5 * 10^4]$.
- 10^6 <= Node.val, insertVal <= 10^6

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given a node in a sorted circular linked list, insert a value maintaining sorted order.

Return any node in the resulting list.

The list may be empty (null). Right?"

"A few quick questions:

What are the insertion cases? Normal (prev<=val<=curr), wraparound (at max->min boundary),

or after a full traversal when all values are equal (insert anywhere).

Do we return the original head or any node? Any node — original head is fine."

"Let me walk: 3->4->1->(back to 3), insert 2 -> insert between 1 and 3 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
values = every value of the circular list in an array
insert the new value in sorted position
rebuild the circular list
```

TIME	SPACE
O(n log n)	O(n)

WHY IT IS SLOW

Rebuilds everything. Walk once around the list to the right gap (previous <= value <= next, or at the wrap point where the order drops), then link the new node there: O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) list and sort. We can do one clean O(n) pass.

Three cases as we walk (prev, curr) pairs around the circle:

Case 1 (normal): prev.val <= insertVal <= curr.val — insert between them.

Case 2 (wraparound): prev.val > curr.val (we're at the max->min boundary).

Insert here if insertVal >= prev.val (new max) OR insertVal <= curr.val (new min).

Case 3 (full circle, all equal): went around without inserting — insert anywhere.

Time: O(n). Space: O(1).

Does that make sense before I code it?

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _708_InsertIntoSortedCircularLL:
    def insert(self, head, insertVal):
        new_node = Node(insertVal)
        if not head:
            new_node.next = new_node
            return new_node
        prev, curr = head, head.next
        while True:
            normal = prev.val <= insertVal <= curr.val
            wraparound = prev.val > curr.val and (insertVa
l >= prev.val or insertVal <= curr.val)
            if normal or wraparound:
                break
            prev, curr = curr, curr.next
            if prev is head:
                break
        prev.next = new_node
        new_node.next = curr
        return head
```

PHASE 5 TEST & DEBUG

"Let me trace the pointer rewiring steps and show how the list structure stays consistent."

3->4->1->3, insert 2: prev=3,curr=4: 3<=2<=4? No. prev=4,curr=1: 4>1 wraparound: 2>=4? No. 2<=1? No.

prev=1,curr=3: 1<=2<=3? Yes -> insert between 1 and 3. -> 3->4->1->2->3 ok

Insert 5 (new max): prev=4,curr=1: wraparound, 5>=4? Yes -> insert between 4 and 1. ok

Insert 0 (new min): prev=4,curr=1: wraparound, 0<=1? Yes -> insert between 4 and 1. ok

The wraparound branch handles both new maximum and new minimum in one check

PHASE 5 TEST & DEBUG

WHY: pointer moves preserve the list's integrity; each rewiring step only changes local next-links while keeping the head/reference valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) at most one full traversal	**O(1)**

All-equal list: we complete the circle without inserting -> break at 'if prev is head' -> insert anywhere.

Follow-up: delete from sorted circular linked list — find and unlink the node.

Follow-up: merge two sorted circular linked lists — split both at the max->min boundary, merge, relink.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Linked List | LeetCode #1171 | 22 of 25 | 22/25

Remove Zero Sum Consecutive Nodes from Linked List

QUESTION

Given the head of a linked list, we repeatedly delete consecutive sequences of nodes that sum to 0 until there are no such sequences.

After doing so, return the head of the final linked list. You may return any such answer.

(Note that in the examples below, all sequences are serializations of ListNode objects.)

Example 1

Input	`head = [1,2,-3,3,1]`

Output	`[3,1]`
	`Note:`
	`The answer [1,2,1] would also be accepted.`

Example 2

Input	`head = [1,2,3,-3,4]`
Output	`[1,2,4]`

Example 3

Input	`head = [1,2,3,-3,-2]`
Output	`[1]`

CONSTRAINTS

- The given linked list will contain between 1 and 1000 nodes.
- Each node in the linked list has -1000 <= node.val <= 1000.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Repeatedly remove consecutive sequences of nodes that sum to 0 until none remain.

Return the resulting linked list. Right?"

"A few quick questions:

Can multiple non-overlapping zero-sum sequences exist? Yes — remove all.

Can values be negative? Yes — that's what makes zero-sum possible.

Does the answer need to be unique? The problem guarantees any valid answer is accepted."

"Let me walk: [1,2,-3,3,1] -> [1,2,-3] sums to 0 -> [3,1] ok.

Also valid: [2,-3,3]=2 (not zero) -> only [1,2,-3] works first.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
while True:
    look for any consecutive run of nodes whose
        values sum to 0 (check every start and
        end) and delete it
    if none is left:
        break
```

TIME	SPACE
O(n^3)	O(1)

WHY IT IS SLOW

Rechecks all runs after each deletion. Two passes with prefix sums in a hash map (the last node for each prefix sum): a repeated prefix sum means the nodes in between sum to zero and can be skipped: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the nested scan — we recompute prefix sums from scratch each pass.

Use a prefix sum + hash map in two passes (O(n)):

Pass 1: walk the list computing running sum; store prefix_sum -> latest node.

If the same prefix sum appears twice, nodes between those two points sum to 0.

The override ensures we store the LAST node with that prefix sum.

Pass 2: walk again; each node's prefix sum is already in the map pointing to the node

just past the zero-sum segment — skip directly to it.

Time: O(n). Space: O(n).

Does that make sense before I code it?

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1171_RemoveZeroSumNodes:
    def removeZeroSumSublists(self, head):
        dummy = ListNode(0, head)
        prefix_to_node = {}
        cur, total = dummy, 0
        while cur:
            total += cur.val
            prefix_to_node[total] = cur
            cur = cur.next
        cur, total = dummy, 0
        while cur:
            total += cur.val
            cur.next = prefix_to_node[total].next
            cur = cur.next
        return dummy.next
```

PHASE 5 TEST & DEBUG

"Let me trace the pointer rewiring steps and show how the list structure stays consistent."

dummy->1->2->-3->3->1:

Pass1: sums: 0->dummy, 1->1, 3->2, 0->-3(override!), 3->3(override!), 4->1.

seen={0:-3node, 1:1node, 3:3node, 4:1node}.

Pass2: total=0->dummy.next=seen[0].next=3node. total=3->3.next=seen[3].next=1node.

total=4->1.next=seen[4].next=None. -> [3,1] ok

Override in pass 1 ensures we skip to the node AFTER the last zero-sum segment

WHY: pointer moves preserve the list's integrity; each rewiring step only changes local next-links while keeping the head/reference valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
two linear passes	prefix sum map

Same pattern as Subarray Sum Equals K (#560) applied to a linked list.

The override-on-collision in pass 1 is the key non-obvious step — worth explaining clearly.

Follow-up: count removed nodes — track length before and after.

Follow-up: if values can be floats, use epsilon comparison for the zero-sum check.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Linked List | LeetCode #1669 | 23 of 25 | 23/25

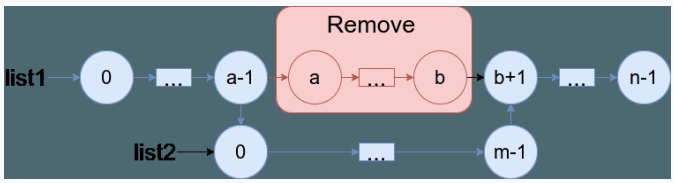
Merge In Between Linked Lists

QUESTION

You are given two linked lists: list1 and list2 of sizes n and m respectively.

Remove list1's nodes from the ath node to the bth node, and put list2 in their place.

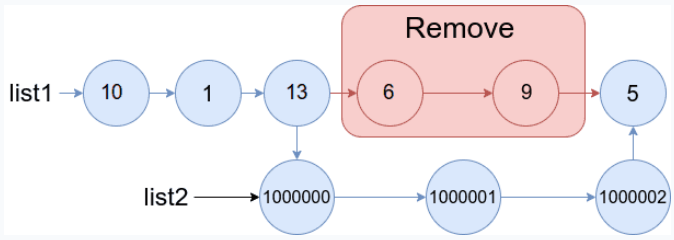
The blue edges and nodes in the following figure indicate the result:



Build the result list and return its head.

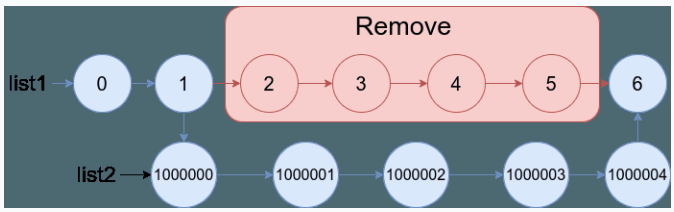
Example 1

Input	`list1 = [10,1,13,6,9,5], a = 3, b = 4, list2 = [1000000,1000001,1000002]`
Output	`[10,1,13,1000000,1000001,1000002,5]`
Explanation	*We remove the nodes 3 and 4 and put the entire list2 in their place. The blue edges and nodes in the above figure indicate the result.*



Example 2

Input	`list1 = [0,1,2,3,4,5,6], a = 2, b = 5, list2 = [1000000,1000001,1000002,1000003,1000004]`
Output	`[0,1,1000000,1000001,1000002,1000003,1000004,6]`
Explanation	*The blue edges and nodes in the above figure indicate the result.*



CONSTRAINTS

- 3 <= list1.length <= 10^4
- 1 <= a <= b < list1.length - 1
- 1 <= list2.length <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Merge list2 into list1 by replacing nodes a..b (inclusive) in list1 with all of list2. Right?"

"A few quick questions:

0-indexed positions? Yes. list2 is appended between node a-1 and node b+1? Yes."

"Let me walk: list1=[0,1,2,3,4,5], a=3, b=4, list2=[1000000,1000001,1000002] -> [0,1,2,1000000,1000001,1000002,5] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
put both lists in arrays
build the new array: the first a items of list1, then
    all of list2, then list1 from b + 1 on
relink the nodes
```

TIME	SPACE
O(n + m)	O(n + m)

WHY IT IS SLOW

Linear, but copies everything. Walk to node a - 1 and node b + 1, then point the first at the head of list2 and the tail of list2 at the second: O(n + m) with O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Find tail of list2, then rewire pointers — O(n). Walk-through:"

Step 1: Walk list1 to find node at index 'a-1' (node just before removal start) -> call it prev_a.

Step 2: Continue walking to find node at index 'b' -> call it after_b (= node after removal end).

Step 3: Find the tail of list2 by walking to its last node -> tail2.

Step 4: Rewire:

`prev_a.next = list2 (connect list1's 'a-1' node to the start of list2).`

`tail2.next = after_b (connect list2's tail to the remaining list1).`

WHY find tail2: we need to connect list2's END to list1's continuation.

Nodes from index a to b are orphaned (no pointer to them).

Time: O(n + m). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1669_MergeInBetweenLinkedLists:
    def mergeInBetween(self, list1, a, b, list2):
        cur = list1
        for _ in range(a - 1):
            cur = cur.next
        node_before_a = cur
        cur = cur.next
        for _ in range(b - a + 1):
            cur = cur.next
        node_after_b = cur
        node_before_a.next = list2
        tail2 = list2
        while tail2.next:
            tail2 = tail2.next
        tail2.next = node_after_b
        return list1
```

PHASE 5 TEST & DEBUG

"Let me trace mergeInBetween(list1=[1,2,3,4,5], a=1, b=3, list2=[1000000,1000001,1000002])."

Find node before index a (index 0): prev=node(1) (the node at index 0... index a-1)

Find node at index b (index 3): node(4). Its next=node(5).

Link: prev.next = head of list2. Tail of list2 .next = node(5)

Wait: prev = node at index a-1=0 -> node(1). node_b_next = node at index b+1=4 -> node(5)

Connect: prev.next = list2_head(1000000); list2_tail(1000002).next = node(5)

Result: 1->1000000->1000001->1000002->5 ok

WHY find a-1 and b+1: cut out nodes a..b, splice in list2.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n+m) O(n) to find splice points + O(m) to find list2 tail	**O(1)**

Follow-up: if list2 has a known tail pointer, skip the tail traversal.

Follow-up: Insert Linked List In Linked List — generalise to any interval.

MEDIUM | Linked List | LeetCode #2095 | 24 of 25 | 24/25

Delete the Middle Node of a Linked List

QUESTION

You are given the head of a linked list. Delete the middle node, and return the head of the modified linked list.

The middle node of a linked list of size n is the n / 2th node from the start using 0-based indexing, where x denotes the largest integer less than or equal to x.

CONSTRAINTS

- For n = 1, 2, 3, 4, and 5, the middle nodes are 0, 1, 1, 2, and 2, respectively.

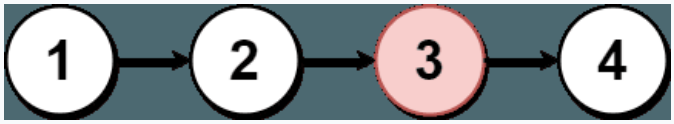
Example 1

Input	`head = [1,3,4,7,1,2,6]`
Output	`[1,3,4,1,2,6]`
Explanation	*The above figure represents the given linked list. The indices of the nodes are written below. Since n = 7, node 3 with value 7 is the middle node, which is marked in red. We return the new list after removing this node.*



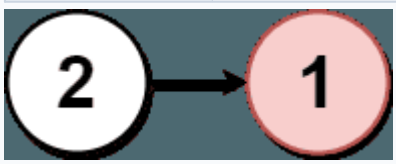
Example 2

Input	`head = [1,2,3,4]`
Output	`[1,2,4]`
Explanation	*The above figure represents the given linked list. For n = 4, node 2 with value 3 is the middle node, which is marked in red.*



Example 3

Input	`head = [2,1]`
Output	`[2]`
Explanation	*The above figure represents the given linked list. For n = 2, node 1 with value 1 is the middle node, which is marked in red. Node 0 with value 2 is the only node remaining after removing node 1.*



- The number of nodes in the list is in the range [1, 10^5].

QUESTION

- 1 <= Node.val <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Delete the middle node of the linked list. If even length, delete the second middle. Right?"

"A few quick questions:

Middle = floor(n/2)? Yes. Return head after deletion?"

"Let me walk: [1,3,4,7,1,2,6] -> delete 7 (index 3) -> [1,3,4,1,2,6] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
count the nodes
walk to the node just before the middle (index n / 2
    - 1) and unlink the next node
```

TIME	SPACE
O(n)	O(1)

WHY IT IS SLOW

Two passes. A slow pointer (one step) and a fast pointer (two steps) with a trailing previous pointer find the middle in a single pass.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Find middle + delete — O(n). Walk-through:"

Slow/fast pointers to find middle:

`slow = fast = head. prev = None.`

While fast and fast.next: prev=slow; slow=slow.next; fast=fast.next.next.

Delete slow (middle): prev.next = slow.next.

Return head.

WHY slow/fast: fast moves 2x speed of slow. When fast reaches end, slow is at middle.

For even-length lists: slow lands at the first of the two middle nodes; delete it.

WHY track prev: singly linked list — to delete slow, must update prev.next.

Edge case: single node -> no prev -> return None.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _2095_DeleteMiddleNodeLinkedList:
    def deleteMiddle(self, head):
        if not head or not head.next:
            return None
        slow = head
        fast = head.next.next
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        slow.next = slow.next.next
        return head
```

PHASE 5 TEST & DEBUG

"Let me trace the pointer rewiring steps and show how the list structure stays consistent."

[1,3,4,7,1,2,6] (n=7): mid=3 (0-indexed, delete index 3=node(7)).

fast starts at head.next.next=4. slow=1.

iter1: slow=3,fast=1. iter2: slow=4,fast=6(end). fast.next=None->stop.

slow=node(4). slow.next=node(1). -> [1,3,4,1,2,6] ok

[1,2] (n=2): fast=None->loop never enters. slow=node(1). 1.next=None. -> [1] ok

Fast=head.next.next offset makes slow land just before middle

WHY: pointer moves preserve the list's integrity; each rewiring step only changes local next-links while keeping the head/reference valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
one pass	two pointers

Follow-up: delete node at specific position from end — same pointer trick with gap.

Follow-up: Middle of Linked List (#876) — find middle without deletion.

MEDIUM | Linked List | LeetCode #2807 | 25 of 25 | 25/25

Insert Greatest Common Divisors in Linked List

QUESTION

Given the head of a linked list head, in which each node contains an integer value.

Between every pair of adjacent nodes, insert a new node with a value equal to the greatest common divisor of them.

Return the linked list after insertion.

The greatest common divisor of two numbers is the largest positive integer that evenly divides both numbers.

Example 1

Input	`head = [18,6,10,3]`
Output	`[18,6,6,2,10,1,3]`
Explanation	*The 1st diagram denotes the initial linked list and the 2nd diagram denotes the linked list after inserting the new nodes (nodes in blue are the inserted nodes).* *- We insert the greatest common divisor of 18 and 6 = 6 between the 1st and the 2nd nodes.* *- We insert the greatest common divisor of 6 and 10 = 2 between the 2nd and the 3rd nodes.* *- We insert the greatest common divisor of 10 and 3 = 1 between the 3rd and the 4th nodes.* *There are no more adjacent nodes, so we return the linked list.*

Example 2

Input	`head = [7]`
Output	`[7]`
Explanation	*The 1st diagram denotes the initial linked list and the 2nd diagram denotes the linked list after inserting the new nodes.* *There are no pairs of adjacent nodes, so we return the initial linked list.*

CONSTRAINTS

- The number of nodes in the list is in the range [1, 5000].
- 1 <= Node.val <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Between every pair of adjacent nodes, insert a new node containing gcd(node.val, node.next.val).

Return the modified head. Right?"

"A few quick questions:

The list could have just one node — return it unchanged? Yes."

PHASE 1 CLARIFY

"Let me walk: head=[18,6,10,3] -> [18,6,6,2,10,1,3] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for b) in b)s:
    g = the largest d from min(a, b) down to 1
        that divides both
    insert g between them
```

TIME	SPACE
O(n * min(a, b))	O(1)

WHY IT IS SLOW

Tries every divisor. Euclid's algorithm (gcd(a, b) = gcd(b, a mod b)) takes O(log min(a, b)) steps.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Single scan — insert node between every pair. O(n). Walk-through:"

`curr = head. While curr.next exists:`

`g = gcd(curr.val, curr.next.val)`

`new_node = ListNode(g)`

`new_node.next = curr.next <- new node points to what was curr's next`

`curr.next = new_node <- curr now points to new node`

`curr = new_node.next <- advance past the new node to the original next`

WHY advance by 2: after inserting, curr.next is the new GCD node; we skip it

and go to the original next pair to process.

Return head (unchanged — we only insert between existing nodes).

Time: O(n * log(max_val)) for GCD calls. Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
import math
class _2807_InsertGreatestCommonDivisorsInLinkedList:
    def insertGreatestCommonDivisors(self, head):
        current = head
        while current.next:
            gcd_value = math.gcd(current.val, current.next
.val)
            new_node = ListNode(gcd_value, current.next)
            current.next = new_node
            current = new_node.next
        return head
```

PHASE 5 TEST & DEBUG

"Let me trace the pointer rewiring steps and show how the list structure stays consistent."

head=[18,6,10,3]: current=18. gcd(18,6)=6. insert 6 between -> [18,6,6,10,3]. current advances to second 6.

```
gcd(6,10)=2. insert 2 -> [18,6,6,2,10,3]. current
  advances to 10.
gcd(10,3)=1. insert 1 -> [18,6,6,2,10,1,3]. current
  advances to 3. current.next is None -> loop ends.
```

-> [18,6,6,2,10,1,3] ok

Advancing current to new_node.next (not new_node) correctly skips past the just-inserted node to the original next pair

WHY: pointer moves preserve the list's integrity; each rewiring step only changes local next-links while keeping the head/reference valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log(max_val))	**O(n)** for new nodes

Follow-up: Merge Two Sorted Lists (#21) — different list manipulation, same splicing technique.

Follow-up: if asked to do it in reverse order or only at even positions, adjust the traversal condition.